

Clonage système & mise à jour multi-disques bootables

Rapport technique complet illustré — hôte riemann@zeta-lab

Disque interne

Seagate
ST1000LM035
1 To — Ubuntu
(système principal)
`/dev/sda`

Disque externe #1

Toshiba
MQ04ABD200
2 To USB3 —
Clone Ubuntu
bootable
`/dev/sdb`

Disque externe #2

Samsung M2
Portable
500 Go USB3 —
Clone Kali
bootable
`/dev/sdc` →
`sdb`

Périmètre exclu : SSD Micron 1100 (boîtier Verbatim), ancien Windows/BitLocker — non touché

Document généré le 21 juin 2026 — édition enrichie (captures intégrées)

Projet : sauvegarde disque résiliente — système d'exploration de l'Hypothèse de Riemann

Sommaire

1. Contexte et objectifs du projet
2. Inventaire des disques et corrections d'identification
3. Phase A — Préparation et installation des outils
4. Phase B — Clonage Kali (Toshiba → Samsung)
5. Phase C — Clonage Ubuntu (sda → Toshiba)
6. Phase D — Dépannage boot Kali sur Samsung
7. Phase E — Première mise à jour système du clone Kali
8. Synthèse de tous les problèmes rencontrés et corrections
9. État final et tableau de bord
10. Leçons apprises et bonnes pratiques retenues
11. Annexe — Galerie complète des captures d'écran

1. Contexte et objectifs du projet

L'objectif initial était de produire trois disques indépendamment bootables à partir d'un poste de travail Linux (machine `zeta-lab`, utilisateur `riemann`) utilisé notamment pour le projet de recherche sur l'hypothèse de Riemann (calcul des zéros non triviaux de la fonction zêta) :

- **Disque interne (`/dev/sda`)** — Ubuntu, système principal de travail.
- **Disque externe Toshiba 2 To USB3** — clone complet et bootable de l'Ubuntu local, destiné à servir de sauvegarde de secours en cas de panne du disque interne.
- **Disque externe Samsung 500 Go USB3** — clone complet et bootable d'une installation Kali Linux préexistante, trouvée sur le Toshiba avant sa réaffectation.

Disque explicitement exclu

Un SSD Micron (boîtier Verbatim) contenant une ancienne installation Windows protégée par BitLocker a été identifié dès l'inventaire initial et volontairement exclu de toute opération de clonage ou de formatage.

Consignes de travail appliquées tout au long du projet

- Commandes fournies prêtes à copier-coller, exécutées par l'utilisateur lui-même.
- Confirmation explicite demandée avant toute action destructrice (formatage, partitionnement, `wipefs`).
- Captures d'écran demandées en cas de doute ou d'erreur visuelle (BIOS, menus GRUB, terminal).
- Tableau d'avancement et liste du reste à faire tenus à jour à chaque étape.
- Toute erreur rencontrée corrigée immédiatement, sans la reproduire à l'étape suivante.

Note sur cette édition

Cette version du rapport intègre directement, dans chaque section concernée, les captures d'écran correspondantes (BIOS, kernel panic, menus GRUB, terminaux) plutôt que de s'y référer uniquement par le texte. Une annexe galerie (§11) regroupe en outre l'intégralité des captures disponibles, dans l'ordre chronologique.

2. Inventaire des disques et corrections d'identification

La toute première étape critique a été d'identifier précisément la nature réelle de chaque disque, car les hypothèses de départ de l'utilisateur (tailles, type SSD/HDD) se sont révélées partiellement inexactes après inspection avec `lsblk`, `smartctl` et `parted`.

Nom au démarrage	Modèle réel identifié	Taille réelle	Contenu d'origine	Rôle final assigné
<code>/dev/sda</code>	Seagate ST1000LM035 (HDD)	931 GiB (1 To)	Ubuntu — système actif	Disque principal (inchangé)
Toshiba (vu un temps comme <code>sdb</code>)	Toshiba MQ04ABD200 (HDD)	2 000 GB (≈1,82 TiB)	Ancienne installation Kali Linux	Reformaté → clone Ubuntu bootable
Samsung (vu un temps comme <code>sdC</code>)	Samsung M2 Portable	500 GB	Ancienne installation Ubuntu inutilisée	Reformaté → clone Kali bootable
Boîtier Verbatim	SSD Micron 1100 (MTFDDAK2)	238 GiB (256 GB réel)	Windows + BitLocker + partition RE	Exclu — non touché

⚠ Problème rencontré — Hypothèse de taille erronée

L'utilisateur pensait disposer d'un SSD externe de 2 To pour le clone Ubuntu ; l'inspection a révélé que le disque réellement câblé à ce moment-là (boîtier Verbatim) ne faisait que **256 Go**, et contenait en réalité une ancienne installation Windows/BitLocker.

```

riemann@zeta-lab:~$ lsblk -l /dev/sd*
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 931,5G 0 disk
sda1 8:1 0 74,5G 0 part /
sda2 8:2 0 1G 0 part /boot/efi
sda3 8:3 0 186,3G 0 part /home
sda4 8:4 0 662,2G 0 part /mnt/data
sda5 8:5 0 7,5G 0 part
sdb 8:16 0 238,5G 0 disk
sdb1 8:17 0 1G 0 part
sdb2 8:18 0 128M 0 part
sdb3 8:19 0 235,3G 0 part
sdb4 8:20 0 2G 0 part
riemann@zeta-lab:~$ lsblk -f /dev/sd*
NAME FSTYPE FSVER LABEL UUID FSAVAIL FSUSE% MOUNTPOINTS
sda
├─sda1 ext4 1.0 2deda3f8-8a82-4b98-af3d-4d32a20e58c3 32,6G 50% /
├─sda2 vfat FAT32 5930-9576 1G 1% /boot/efi
├─sda3 ext4 1.0 f0685a71-e8cf-45ea-a8bc-1395b575ec2f 59,7G 62% /home
├─sda4 ext4 1.0 e5a5e690-c038-4198-bdc7-6f7431d0635b 589,7G 4% /mnt/data
└─sda5 swap 1 1e353ef5-4a66-45af-9ca2-b298072dfd38
sdb1 ext4 1.0 2deda3f8-8a82-4b98-af3d-4d32a20e58c3 32,6G 50% /
sdb2 vfat FAT32 5930-9576 1G 1% /boot/efi
sdb3 ext4 1.0 f0685a71-e8cf-45ea-a8bc-1395b575ec2f 59,7G 62% /home
sdb4 ext4 1.0 e5a5e690-c038-4198-bdc7-6f7431d0635b 589,7G 4% /mnt/data
sdb5 swap 1 1e353ef5-4a66-45af-9ca2-b298072dfd38
sdb
├─sdb1 vfat FAT32 SYSTEM E200-BA21
├─sdb2
├─sdb3 BitLocker 2
└─sdb4 ntfs RE E29A395F9A393201
sdb1 vfat FAT32 SYSTEM E200-BA21
sdb2
sdb3 BitLocker 2
sdb4 ntfs RE E29A395F9A393201
riemann@zeta-lab:~$

```

Capture — premier `lsblk -l /dev/sd*` et `lsblk -f`. Le disque branché en `sdb` affiche 238,5G avec une partition `BitLocker` : c'est le SSD Micron, pas le disque de 2 To attendu.

✓ Correction appliquée

Réinventaire complet de tous les disques connectés avec `lsblk -f /dev/sd*` et `smartctl`. Le véritable disque 2 To (Toshiba) et le disque 500 Go (Samsung) ont été identifiés séparément, et les rôles ont été réattribués en conséquence : Toshiba pour le clone Ubuntu (volume suffisant), Samsung pour le clone Kali (après nettoyage du volumineux dossier de dumps forensiques, voir §4).

```

riemann@zeta-lab:~$ lsblk -f /dev/sd*
NAME FSTYPE FSVER LABEL UUID FSAVAIL FSUSE% MOUNTPOINTS
sda
├─sda1 ext4 1.0 2deda3f8-8a82-4b98-af3d-4d32a20e58c3 32,8G 50% /
├─sda2 vfat FAT32 5930-9576 1G 1% /boot/efi
├─sda3 ext4 1.0 f0685a71-e8cf-45ea-a8bc-1395b575ec2f 59,6G 62% /home
├─sda4 ext4 1.0 e5a5e690-c038-4198-bdc7-6f7431d0635b 589,7G 4% /mnt/data
└─sda5 swap 1 1e353ef5-4a66-45af-9ca2-b298072dfd38
sdb1 vfat FAT32 2257-5D4C 1010,6G 40% /media/riemann/2657601b-dd77-4569-b535-bb2b788db71f
├─sdb2 ext4 1.0 d599a81f-4ba1-454b-a636-67a061015938
├─sdb3 swap 1 2257-5D4C
└─sdb4 ext4 1.0 d599a81f-4ba1-454b-a636-67a061015938
sdb2 vfat FAT32 2257-5D4C 1010,6G 40% /media/riemann/2657601b-dd77-4569-b535-bb2b788db71f
sdb3 swap 1 d599a81f-4ba1-454b-a636-67a061015938
sdb4 ext4 1.0 d599a81f-4ba1-454b-a636-67a061015938
sdc
├─sdc1 vfat FAT32 9D0C-DC04 422,6G 2% /media/riemann/3afdd771-9528-4b5f-adcf-48277dfad5ba
├─sdc2 ext4 1.0 3afdd771-9528-4b5f-adcf-48277dfad5ba 422,6G 2% /media/riemann/3afdd771-9528-4b5f-adcf-48277dfad5ba
└─sdc3 swap 1 3afdd771-9528-4b5f-adcf-48277dfad5ba
riemann@zeta-lab:~$

```

Capture — réinventaire correct. `lsblk -f /dev/sd*` révèle `sdb` = Toshiba (1010,6G, monté avec le Kali d'origine, label UUID `2657601b...`) et `sdc` = Samsung (422,6G). Les bons disques sont désormais identifiés sans ambiguïté.

Règle retenue pour la suite du projet

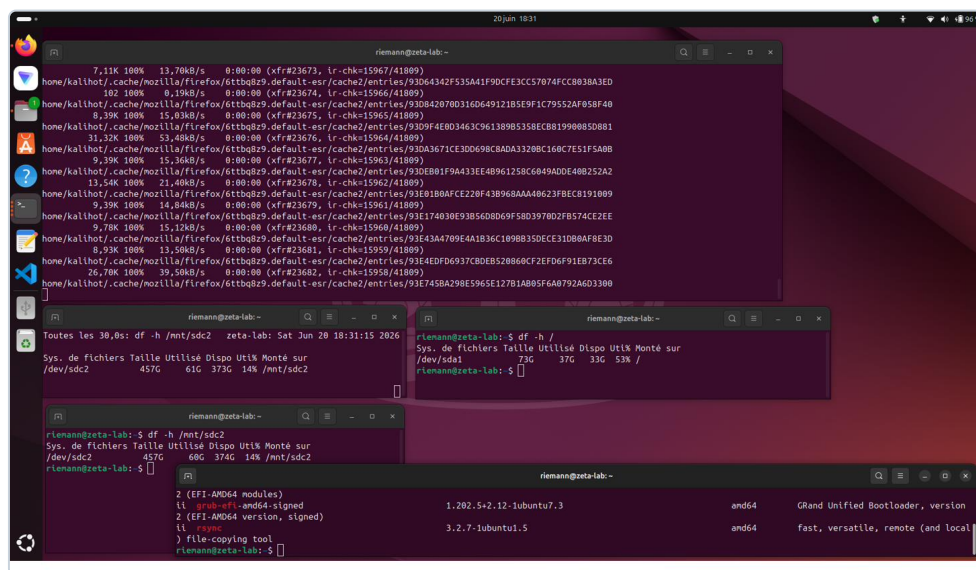
Les noms `/dev/sdX` sont instables d'une session à l'autre selon l'ordre de branchement USB. Toute identification a ensuite été faite par label (`kali-clone` , `root-clone`) ou par UUID, jamais par lettre seule.

3. Phase A — Préparation et installation des outils

Avant tout clonage, l'outil principal nécessaire a été vérifié sur le système source (Ubuntu, `sda`) :

```
sudo apt update
sudo apt install rsync
```

`rsync` était déjà installé en version récente (3.2.7-1ubuntu1.5) — aucune installation supplémentaire n'a donc été nécessaire à cette étape. Les autres outils utilisés (`parted`, `wipefs`, `mkfs.ext4`, `mkfs.vfat`, `mkswap`, `grub-install`, `update-grub`, `update-initramfs`, `smartctl`) étaient également déjà présents sur le système Ubuntu de base.



Capture — vérification des paquets installés. En bas à droite : `grub-efi-amd64-signed`

(1.202.5+2.12-1ubuntu7.3) et `rsync` (3.2.7-1ubuntu1.5) confirmés présents et à jour, pendant que le clonage Kali est déjà en cours dans les autres terminaux.

Contrôle disque rapide (SMART)

Un contrôle de santé rapide a été effectué sur les disques cibles avant toute opération destructrice :

```
sudo smartctl -H /dev/sdb
sudo smartctl -H /dev/sdc
```

Résultat : **PASSED** pour le disque Toshiba. Le contrôle du Samsung a été jugé non bloquant et traité comme optionnel à ce stade du projet.

4. Phase B — Clonage Kali (Toshiba → Samsung)

Avant de pouvoir réutiliser le Toshiba (2 To) pour le clone Ubuntu, l'installation Kali existante qu'il contenait a d'abord été sauvegardée sur le disque Samsung (500 Go).

4.1 — Analyse du volume de données à copier

Le volume total occupé par l'installation Kali sur le Toshiba était de 721 Go, ce qui dépassait la capacité du Samsung (500 Go / ≈ 423 GiB utilisables). Une analyse fine a permis d'isoler la cause :

Élément	Taille	Décision
<code>home/img</code>	509 Go	Dossier unique, identifié comme dump forensique recréable — exclu
Reste du système Kali	≈ 212 Go	Conservé intégralement — rentre dans les 423 GiB du Samsung avec marge
<code>/home Ubuntu</code> (référence, étape ultérieure)	118 Go	Utile pour dimensionner le clone Ubuntu (Phase C)

✓ Décision validée avec l'utilisateur

`home/img` a été confirmé par l'utilisateur comme une donnée recréable (dump forensique de mission terminée), permettant son exclusion sans perte de valeur réelle.

4.2 — Commande de copie (dry-run puis exécution réelle)

Une simulation a d'abord été lancée pour valider le volume avant tout transfert réel :

```
rsync -avh --dry-run \
  --exclude='home/img/' \
  "/media/riemann/<UUID-toshiba>/" \
  "/media/riemann/<UUID-samsung>/kali-backup/" \
  | tail -40
```

⚠ Problème rencontré — Erreurs de permissions

La première tentative de copie réelle a été lancée en utilisateur normal (`riemann`), provoquant des erreurs de permission sur de nombreux répertoires système sensibles de Kali (`/root` , `/etc/ssl/private` , `/var/lib/mysql/...`), ces fichiers appartenant à `root` et n'étant pas lisibles par un utilisateur standard.

✓ Correction appliquée

Toutes les opérations `rsync` de clonage de systèmes ont ensuite été systématiquement lancées avec `sudo` , avec l'option `--one-file-system` pour respecter les limites de montage partition par partition (une commande `rsync` distincte par partition lors des clonages multi-partitions).

Commande type retenue pour la suite du projet :

```
sudo rsync -avh --progress --one-file-system \
  --exclude='home/img/' \
  --log-file=/home/riemann/clone_kali.log \
  /source/ /destination/
```

```
riemann@zeta-lab:~$ sudo blkid /dev/sdc1 /dev/sdc2
/dev/sdc1: LABEL="FATBOOT"=EFI LABEL="EFI" UUID="7f66-0c9e" BLOCK_SIZE="512" TYPE="vfat" PARTLABEL="ESP" PARTUUID="bf87259d-2893-4876-be16-624180d44e50"
/dev/sdc2: LABEL="kali-clone" UUID="eaa96119-68d1-4a84-811b-c2e212b8b348" BLOCK_SIZE="4096" TYPE="ext4" PARTLABEL="primary" PARTUUID="d9fedb7f-19d9-4281-af62-543d298b646e"
riemann@zeta-lab:~$ sudo mkdir -p /mnt/sdc2
riemann@zeta-lab:~$ sudo mount /dev/sdc2 /mnt/sdc2
riemann@zeta-lab:~$ df -h /mnt/sdc2
Sys. de fichiers Taille Utilisé Dispo Util% Monté sur
/dev/sdc2 457G 28K 434G 1% /mnt/sdc2
total 24
drwxr-xr-x 3 root root 4096 juin 20 17:55 .
drwxr-xr-x 4 root root 4096 juin 20 17:57 ..
drwx----- 2 root root 16384 juin 20 17:55 lost+found
riemann@zeta-lab:~$ sudo rsync -avh --progress \
  --exclude='home/img/' \
  --log-file=/home/riemann/kali-to-sdc-rsync.log \
  "/media/riemann/2657601b-d477-4569-b535-bb2b788db71f" \
  "/mnt/sdc2/"
sending incremental file list
./
bin -> usr/bin
initrd.img -> boot/initrd.img-6.18.12+kali-amd64
initrd.img.old -> boot/initrd.img-6.18.9+kali-amd64
lib -> usr/lib
lib32 -> usr/lib32
lib64 -> usr/lib64
sbin -> usr/sbin
vmlinuz -> boot/vmlinuz-6.18.12+kali-amd64
vmlinuz.old -> boot/vmlinuz-6.18.9+kali-amd64
.cache/
boot/
boot/System.map-6.17.10+kali-amd64
92 100% 0,00kB/s 0:00:00 (xfr#1, lr-chk=1379/1407)
boot/System.map-6.18.12+kali-amd64
92 100% 14,97kB/s 0:00:00 (xfr#2, lr-chk=1378/1407)
boot/System.map-6.18.3+kali-2-amd64
93 100% 3,95kB/s 0:00:00 (xfr#3, lr-chk=1377/1407)
boot/System.map-6.18.5+kali-amd64
91 100% 2,61kB/s 0:00:00 (xfr#4, lr-chk=1376/1407)
boot/System.map-6.18.9+kali-amd64
91 100% 2,17kB/s 0:00:00 (xfr#5, lr-chk=1375/1407)
boot/System.map-6.19.14+kali-amd64
92 100% 1,52kB/s 0:00:00 (xfr#6, lr-chk=1374/1407)
boot/config-6.17.10+kali-amd64
291,88K 100% 3,39MB/s 0:00:00 (xfr#7, lr-chk=1373/1407)
```

Capture — rsync Kali en cours, correctement lancé avec `sudo` . La commande référence bien `--exclude='home/img/'` , un fichier de log dédié, et la source/destination par UUID de montage. Premiers fichiers transférés visibles (initrd, vmlinuz, System.map des 6 noyaux Kali).

```
riemann@zeta-lab:~$ df -h /mnt/sdc2
Sys. de fichiers Taille Utilisé Dispo Util% Monté sur
/dev/sdc2 457G 28K 434G 1% /mnt/sdc2
```

Suivi en double-terminal. Transfert de gros fichiers ISO (`blackarch-linux-full` 4,93G, etc.) en haut ; `df -h /mnt/sdc2` rafraîchi toutes les 30s en bas (6,3G/427G à ce stade).

```
sent 226,30G bytes, received 11,20M bytes 32,58M bytes/sec
total size is 226,30G, compressed 10,14G
```

Fin du rsync Kali. `sent 226,30G bytes received 11,20M bytes 32,58M bytes/sec` — `total size is 226,20G` . Volume cohérent avec les 212 Go estimés (hors `home/img`).

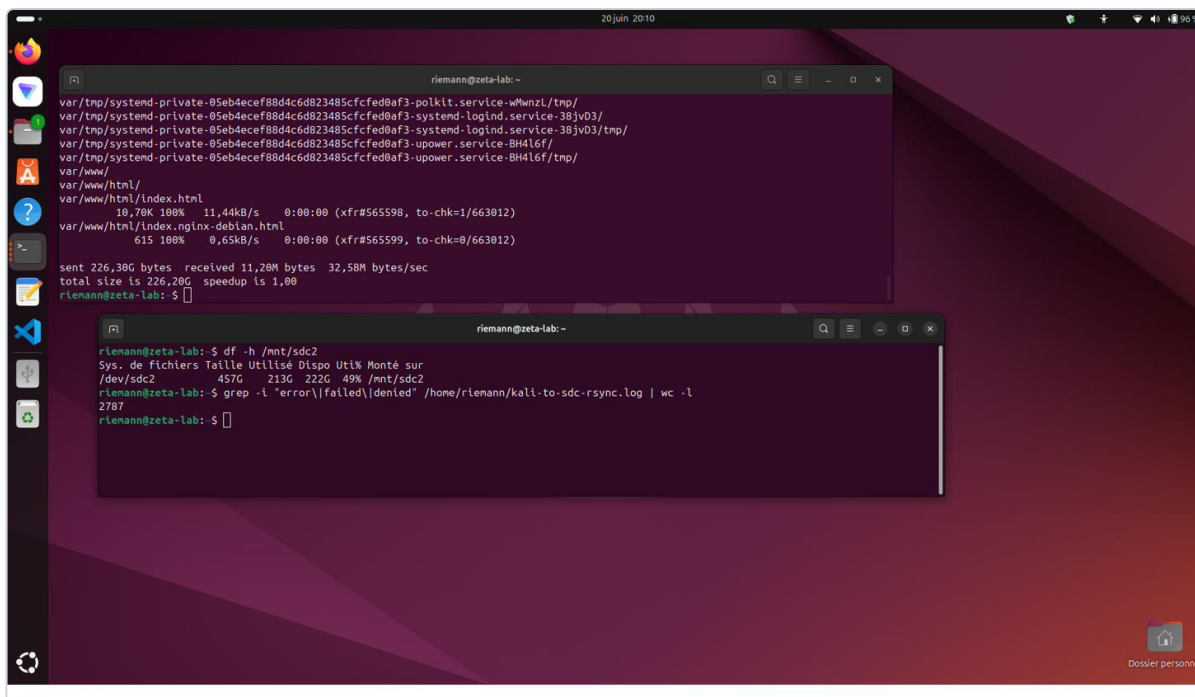
4.3 — Vérification qualité post-transfert

Une fois le transfert terminé, une vérification du journal de copie a été effectuée par précaution, en cherchant les motifs `error` , `failed` et `denied` .

⚠ Alerte apparente — 2787 occurrences suspectes dans le log

```
grep -i "error\\|failed\\|denied" /home/riemann/kali-to-sdc-rsync.log | wc -l
2787
```

Un premier comptage brut du journal de transfert a fait remonter 2787 lignes contenant l'un de ces motifs, ce qui semblait à première vue préoccupant pour un clonage censé être propre.



Capture — comptage initial alarmant. 2787 lignes correspondent au filtre `error|failed|denied` dans le journal `rsync`.

✓ Analyse fine — faux positifs confirmés

Un examen détaillé des lignes correspondantes a montré qu'il s'agissait presque exclusivement de **noms de fichiers légitimes** contenant ces mots dans leur nom (ex. `libpgp-error0`, `scapy/error.py`, `mingw-w64/include/error.h`, `apache2/error.log`), et non de véritables échecs de transfert rsync. Une déduplication par motif (`sed + sort -u`) a permis de le confirmer rapidement.

```
grep -iE "error|failed|denied" /home/riemann/kali-to-sdc-rsync.log \
| sed -E 's/.*\((.*)\).*/\1/' | sort | uniq -c | sort -rn | head -10
grep -c "^rsync:" /home/riemann/kali-to-sdc-rsync.log
0
```

Le test décisif est le comptage des lignes commençant réellement par `rsync:` (préfixe utilisé par rsync pour ses propres messages d'erreur) : **0 résultat**. Aucune erreur de transfert rsync réelle n'a donc été constatée sur ce clonage.

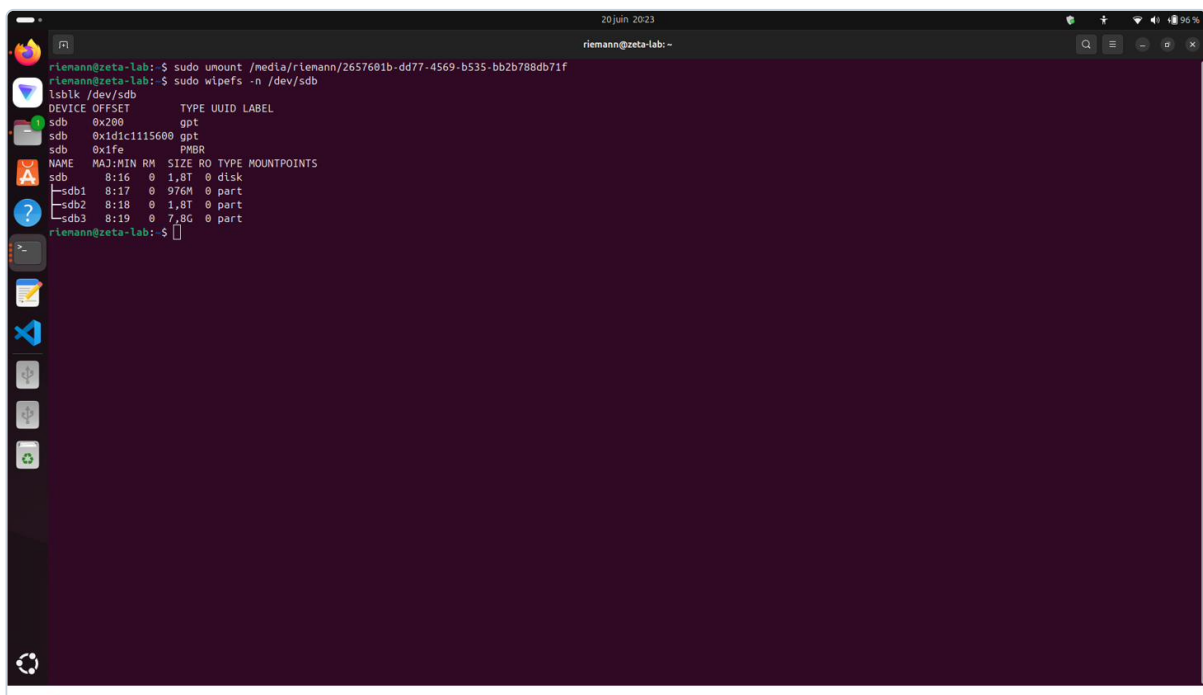
5. Phase C — Clonage Ubuntu (sda → Toshiba)

Une fois le Toshiba libéré du contenu Kali (sauvegardé sur Samsung), il a été entièrement reformaté pour accueillir le clone bootable d'Ubuntu.

5.1 — Effacement et partitionnement (GPT)

Par sécurité, l'effacement de la table de partitions a d'abord été simulé en mode lecture seule avant toute écriture réelle :

```
sudo wipefs -n /dev/sdb # simulation — aucune écriture
```



Capture — `wipefs -n` (dry-run). Affiche les signatures GPT/PMBR qui seraient effacées, sans toucher au disque. Le `lsblk` confirme la structure encore présente du Toshiba (1,8T, 3 partitions héritées de Kali).

Après confirmation explicite de l'utilisateur, l'effacement réel a été exécuté :

```
sudo wipefs -a /dev/sdb
sudo parted /dev/sdb --script mklabel gpt
```

```
riemann@zeta-lab:~$ sudo umount /media/riemann/2657601b-dd77-4569-b535-bb2b788db71f
riemann@zeta-lab:~$ sudo wipefs -n /dev/sdb
lsblk /dev/sdb
  DEVICE OFFSEI          TYPE UUID LABEL
  sdb      0x200          gpt
  sdb      0x1d1c1115600  gpt
  sdb      0x1fe          PMBR
riemann@zeta-lab:~$ sudo wipefs -a /dev/sdb
/dev/sdb : 8 octets ont été effacés à l'index 0x00000200 (gpt) : 45 46 49 20 50 41 52 54
/dev/sdb : 8 octets ont été effacés à l'index 0x1d1c1115600 (gpt) : 45 46 49 20 50 41 52 54
/dev/sdb : 2 octets ont été effacés à l'index 0x000001fe (PMBR) : 55 aa
/dev/sdb : appel d'ioctl pour relire la table de partitions : Succès
riemann@zeta-lab:~$ sudo wipefs -a /dev/sdb
riemann@zeta-lab:~$
```

Capture — **wipefs -a** réel. 8 octets effacés à l'index GPT primaire, 8 octets à l'index GPT secondaire, 2 octets au PMBR. Relecture de la table de partitions confirmée en succès.

⚠ Problème rencontré — Dépassement de capacité dans le calcul des partitions

Lors de la création de la 4^e partition (`/mnt/data`), la valeur de fin `1924409MiB` a été calculée à partir d'une estimation erronée de la capacité totale du disque. Erreur retournée :

```
Erreur: La localisation 1924409MiB est en dehors du périphérique /dev/sdb.
```

Cause : confusion entre la notation décimale (2000GB annoncés par `parted`) et la notation binaire (TiO) utilisée par `lsblk` (1,8T). La partition `/mnt/data` n'a pas pu être créée intégralement, et la partition swap n'a pas pu être créée du tout lors de cette première tentative.

```
riemann@zeta-lab:~$ sudo umount /media/riemann/2657601b-dd77-4569-b535-bb2b788db71f
riemann@zeta-lab:~$ sudo wipefs -n /dev/sdb
lsblk /dev/sdb
  DEVICE OFFSEI          TYPE UUID LABEL
  sdb      0x200             gpt
  sdb      0x1d1c1115600      gpt
  sdb      0x1fe             PMBR
riemann@zeta-lab:~$ sudo wipefs -a /dev/sdb
/dev/sdb : 8 octets ont été effacés à l'index 0x00000200 (gpt) : 45 46 49 20 50 41 52 54
/dev/sdb : 8 octets ont été effacés à l'index 0x1d1c1115600 (gpt) : 45 46 49 20 50 41 52 54
/dev/sdb : 2 octets ont été effacés à l'index 0x000001fe (PMBR) : 55 aa
/dev/sdb : appel d'ioctl pour relire la table de partitions : Succès
riemann@zeta-lab:~$ sudo wipefs -a /dev/sdb
riemann@zeta-lab:~$ sudo parted /dev/sdb --script mklabel gpt
riemann@zeta-lab:~$ sudo parted /dev/sdb --script mkpart ESP fat32 1MiB 1025MiB
sudo parted /dev/sdb --script set 1 esp on
riemann@zeta-lab:~$ sudo parted /dev/sdb --script mkpart primary ext4 1025MiB 102425MiB
riemann@zeta-lab:~$ sudo parted /dev/sdb --script mkpart primary ext4 102425MiB 358425MiB
riemann@zeta-lab:~$ sudo parted /dev/sdb --script mkpart primary ext4 358425MiB 1924409MiB
Erreur: La localisation 1924409MiB est en dehors du périphérique /dev/sdb.
riemann@zeta-lab:~$ sudo parted /dev/sdb --script mkpart primary linux-swap 1924409MiB 100%
Erreur: La localisation 1924409MiB est en dehors du périphérique /dev/sdb.
riemann@zeta-lab:~$
```

Capture — erreur de dépassement capturée en direct. Les deux dernières commandes `mkpart` échouent identiquement : « La localisation 1924409MiB est en dehors du périphérique /dev/sdb ».

✓ Correction appliquée

Relecture de la taille exacte et déjà partitionnée du disque avec :

```
sudo parted /dev/sdb --script print
```

Le disque a été confirmé à exactement 2000GB (notation décimale), avec 376GB déjà utilisés par les 3 premières partitions. La 4^e partition a ensuite été recrée en utilisant **100%** comme borne finale plutôt qu'une valeur MiB calculée à la main — éliminant tout risque de dépassement :

```
sudo parted /dev/sdb --script mkpart ESP fat32 1MiB 1025MiB
sudo parted /dev/sdb --script set 1 esp on
sudo parted /dev/sdb --script mkpart primary ext4 1025MiB 102425MiB
sudo parted /dev/sdb --script mkpart primary ext4 102425MiB 358425MiB
sudo parted /dev/sdb --script mkpart primary ext4 358425MiB 100%
```

Règle retenue

Pour toute partition qui doit occuper le reste de l'espace disponible sur un disque, utiliser systématiquement **100%** comme borne finale plutôt qu'une valeur MiB calculée manuellement, et toujours confirmer la taille réelle avec `parted ... print` avant de lancer des commandes de partitionnement en chaîne.

5.2 — Formatage des partitions

```
sudo mkfs.vfat -F32 /dev/sdb1
sudo mkfs.ext4 -L root-clone /dev/sdb2
sudo mkfs.ext4 -L home-clone /dev/sdb3
sudo mkfs.ext4 -L data-clone /dev/sdb4
```

```
sudo mkswap -L swap-clone /dev/sdb5
sudo swapon /dev/sdb5
```

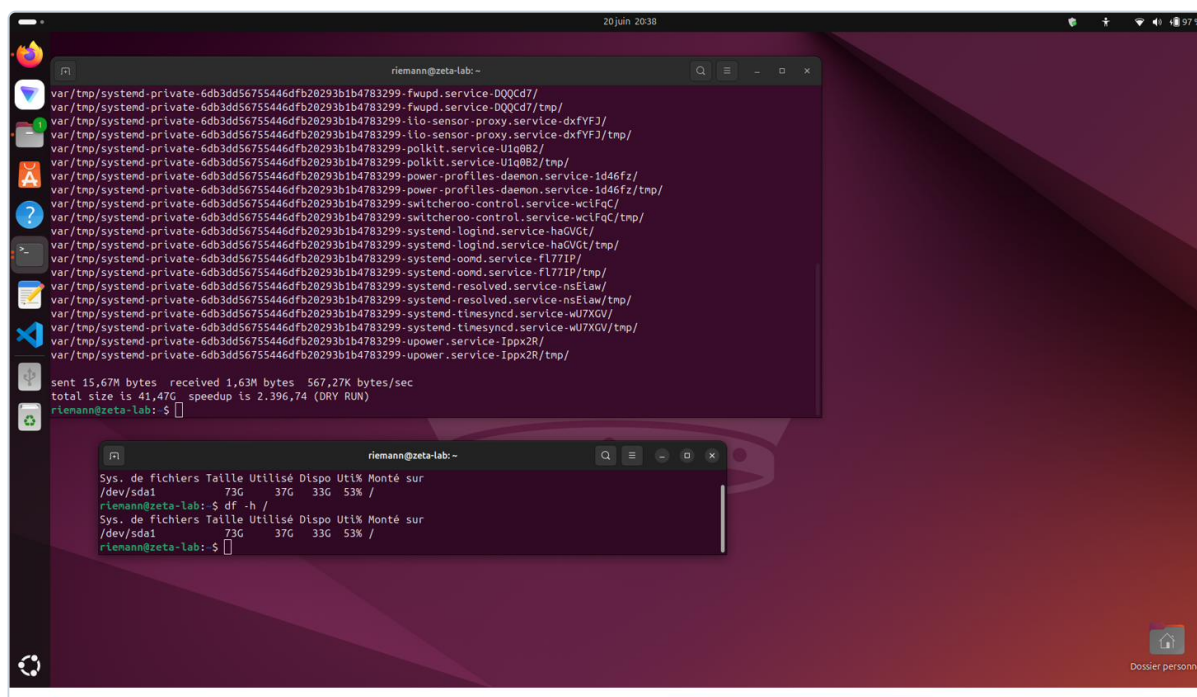
i Précision apportée par les captures — 5 partitions, pas 4

Le partitionnement final du Toshiba comporte en réalité **cinq** partitions et non quatre : en plus de l'EFI (`sdb1`), de la racine (`sdb2` , label `root-clone`), du home (`sdb3` , label `home-clone`) et de `/mnt/data` (`sdb4` , label `data-clone`), une cinquième partition **swap** (`sdb5` , label `swap-clone`) a été créée pour reproduire fidèlement la disposition du disque source `sda` , qui possède lui aussi 5 partitions dont un swap dédié.

5.3 — Clonage des données (rsync multi-partitions)

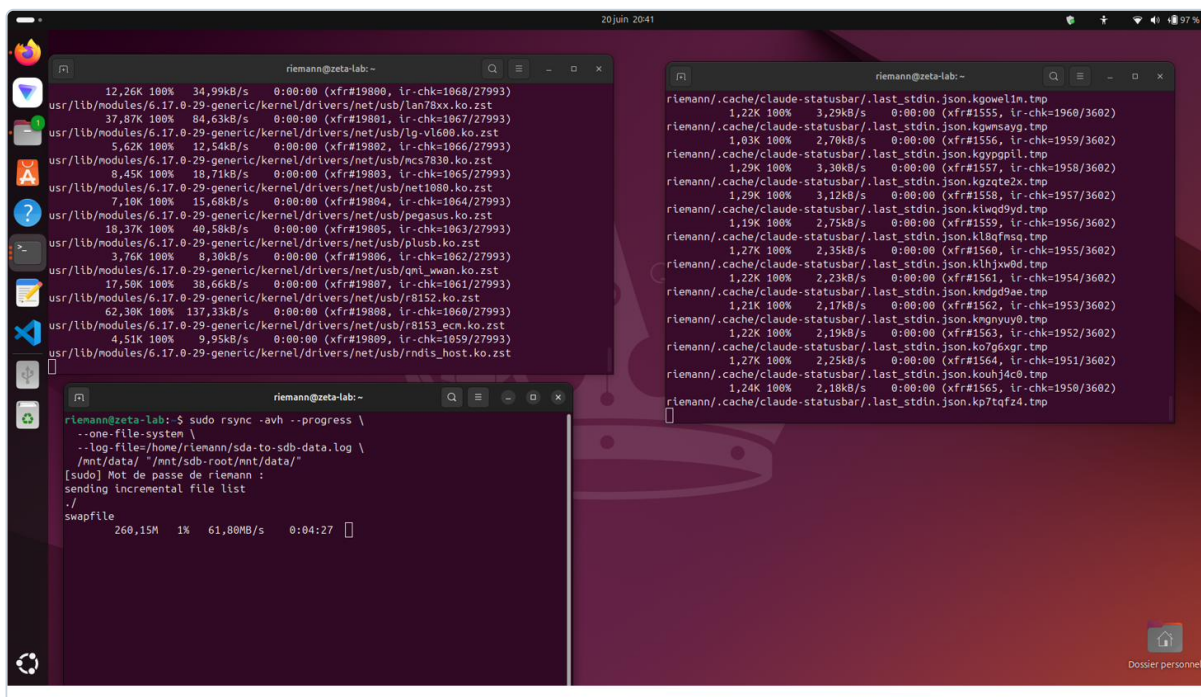
Conformément à la correction retenue en Phase B, chaque partition a été copiée individuellement avec `--one-file-system` , en root. Un essai à blanc a précédé chaque copie réelle :

```
sudo rsync -avh --progress --one-file-system --dry-run / /mnt/clone-root/
```



Capture — `rsync --dry-run` sur la racine. En haut : simulation affichant `total size is 41,47G ... speedup is 2.396,74 (DRY RUN)` , confirmant le volume avant tout transfert réel. En bas : `df -h /` confirme 37G utilisés sur `sda1` (cohérent).

```
sudo rsync -avh --progress --one-file-system / /mnt/clone-root/
sudo rsync -avh --progress --one-file-system /home/ /mnt/clone-root/home/
sudo rsync -avh --progress --one-file-system /mnt/data/ /mnt/clone-root/mnt/data/
```

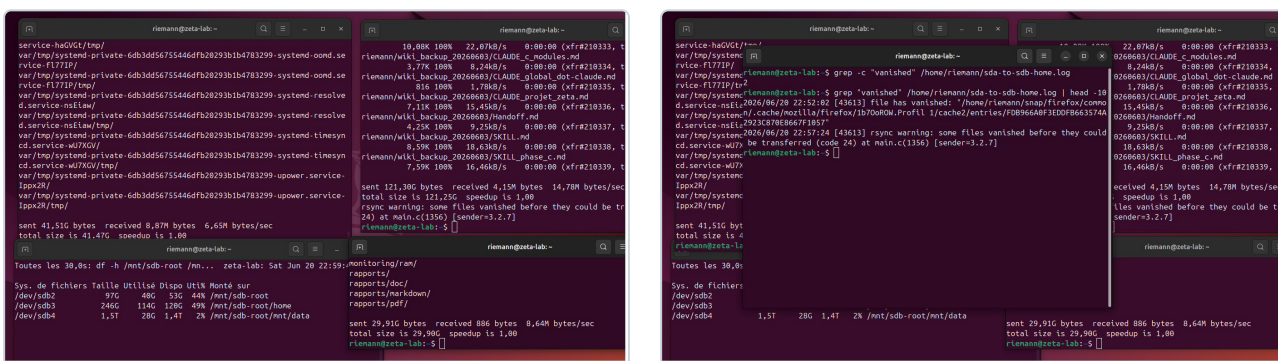


Capture — trois rsync en parallèle. Transfert simultané de la racine (en bas à gauche, démarrage de la copie du `Swapfile`), de `/home` (à droite) et de `/mnt/data` (modules noyau USB en haut à gauche).

⚠️ Avertissement bénin observé — fichiers « vanished »

```
rsync warning: some files vanished before they could be transferred (code 24) at
main.c(1356) [sender=3.2.7]
```

Ce message est apparu sur le transfert de `/home`. Une vérification ciblée a confirmé qu'il s'agissait d'un fichier de cache Firefox temporaire ayant disparu entre l'énumération et la copie effective (système actif en cours d'utilisation pendant le clonage) — comportement strictement normal pour un `rsync` exécuté sur un système live, sans incidence sur l'intégrité du clone.



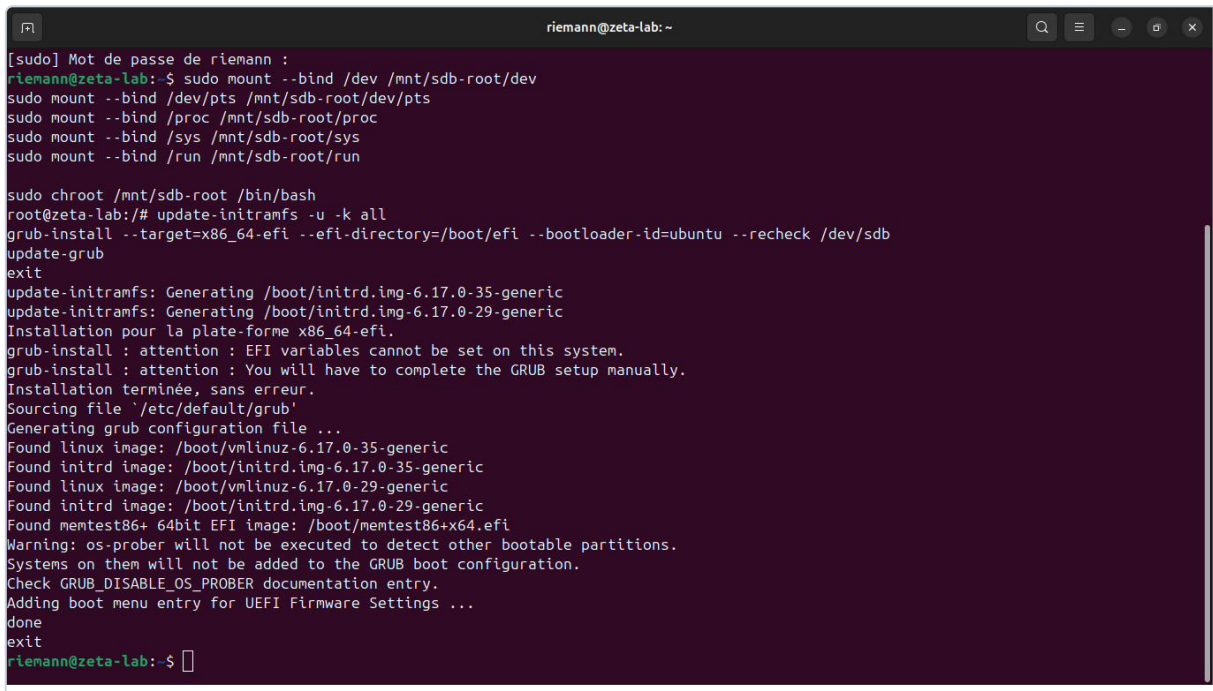
Fin du rsync `/mnt/data`. `sent 41,51G bytes ... total size is 41,47G` ; le warning « vanished » apparaît dans le terminal voisin (transfert de `/home`, 121,30G envoyés).

Vérification du warning. `grep "vanished" /home/riemann/sda-to-sdb-home.log` identifie précisément le fichier disparu : un cache Firefox (`.../cache2/entries/...`) — confirmation du caractère bénin.

5.4 — Installation de GRUB (chroot)

```
sudo mount --bind /dev /mnt/clone-root/dev
sudo mount --bind /dev/pts /mnt/clone-root/dev/pts
sudo mount --bind /proc /mnt/clone-root/proc
sudo mount --bind /sys /mnt/clone-root/sys
sudo mount --bind /run /mnt/clone-root/run

sudo chroot /mnt/clone-root /bin/bash
update-initramfs -u -k all
grub-install --target=x86_64-efi --efi-directory=/boot/efi \
  --bootloader-id=ubuntu --recheck /dev/sdb
update-grub
exit
```



```
riemann@zeta-lab: ~
[sudo] Mot de passe de riemann :
riemann@zeta-lab:~$ sudo mount --bind /dev /mnt/sdb-root/dev
sudo mount --bind /dev/pts /mnt/sdb-root/dev/pts
sudo mount --bind /proc /mnt/sdb-root/proc
sudo mount --bind /sys /mnt/sdb-root/sys
sudo mount --bind /run /mnt/sdb-root/run

sudo chroot /mnt/sdb-root /bin/bash
root@zeta-lab:/# update-initramfs -u -k all
grub-install --target=x86_64-efi --efi-directory=/boot/efi --bootloader-id=ubuntu --recheck /dev/sdb
update-grub
exit
update-initramfs: Generating /boot/initrd.img-6.17.0-35-generic
update-initramfs: Generating /boot/initrd.img-6.17.0-29-generic
Installation pour la plate-forme x86_64-efi.
grub-install : attention : EFI variables cannot be set on this system.
grub-install : attention : You will have to complete the GRUB setup manually.
Installation terminée, sans erreur.
Sourcing file '/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.17.0-35-generic
Found initrd image: /boot/initrd.img-6.17.0-35-generic
Found linux image: /boot/vmlinuz-6.17.0-29-generic
Found initrd image: /boot/initrd.img-6.17.0-29-generic
Found memtest86+ 64bit EFI image: /boot/memtest86+x64.efi
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
exit
riemann@zeta-lab:~$
```

Capture — installation GRUB complète en chroot. Régénération de l'initramfs pour les 2 noyaux Ubuntu, puis `grub-install` et `update-grub` avec détection correcte des images linux/initrd et de l'image memtest86+. Un avertissement `EFI variables cannot be set on this system` apparaît — normal en chroot (le firmware UEFI n'est accessible que depuis le système réellement démarré), sans incidence puisque la configuration GRUB est de toute façon écrite sur le disque.

5.5 — Correction des UUID dans fstab

⚠ Problème rencontré — fstab pointant vers les anciens UUID

Après clonage, le fichier `/etc/fstab` du clone référençait encore les UUID des partitions d'origine (`sda`), inexistantes sur le nouveau disque, ce qui aurait empêché un démarrage correct.

```
riemann@zeta-lab: ~  
Found linux image: /boot/vmlinuz-6.17.0-29-generic  
Found initrd image: /boot/initrd.img-6.17.0-29-generic  
Found memtest86+ 64bit EFI image: /boot/memtest86+x64.efi  
Warning: os-prober will not be executed to detect other bootable partitions.  
Systems on then will not be added to the GRUB boot configuration.  
Check GRUB_DISABLE_OS_PROBER documentation entry.  
Adding boot menu entry for UEFI Firmware Settings ...  
done  
exit  
riemann@zeta-lab:~$ sudo umount /mnt/sdb-root/dev/pts  
sudo umount /mnt/sdb-root/dev  
sudo umount /mnt/sdb-root/proc  
sudo umount /mnt/sdb-root/sys  
sudo umount /mnt/sdb-root/run  
sudo umount /mnt/sdb-root/boot/efi  
riemann@zeta-lab:~$ sudo umount /mnt/sdb-root/dev/pts  
sudo umount /mnt/sdb-root/dev  
sudo umount /mnt/sdb-root/proc  
sudo umount /mnt/sdb-root/sys  
sudo umount /mnt/sdb-root/run  
sudo umount /mnt/sdb-root/boot/efi  
umount: /mnt/sdb-root/dev/pts: Aucun point de montage indiqué.  
umount: /mnt/sdb-root/dev: non monté.  
umount: /mnt/sdb-root/proc: non monté.  
umount: /mnt/sdb-root/sys: non monté.  
umount: /mnt/sdb-root/run: non monté.  
umount: /mnt/sdb-root/boot/efi: non monté.  
riemann@zeta-lab:~$ ls -lh "/mnt/sdb-root/mnt/data/swapfile"  
ls -lh /mnt/data/swapfile  
ls: impossible d'accéder à '/mnt/sdb-root/mnt/data/swapfile': Aucun fichier ou dossier de ce nom  
-rw----- 1 root root 16G avril 11 15:37 /mnt/data/swapfile  
riemann@zeta-lab:~$
```

Capture — fstab du clone avant correction. Les quatre lignes actives référencent encore les UUID de `sda1` à `sda4` (commentaires « was on /dev/sdaX during curtin installation »). Le `swapfile` n'est, à ce stade, pas encore accessible depuis le point de montage du clone.

✓ Correction appliquée

Remplacement des anciens UUID par les nouveaux (relevés via `blkid`) avec `sed -i` :

```
sudo blkid /dev/sdb1 /dev/sdb2 /dev/sdb3 /dev/sdb4 /dev/sdb5
```

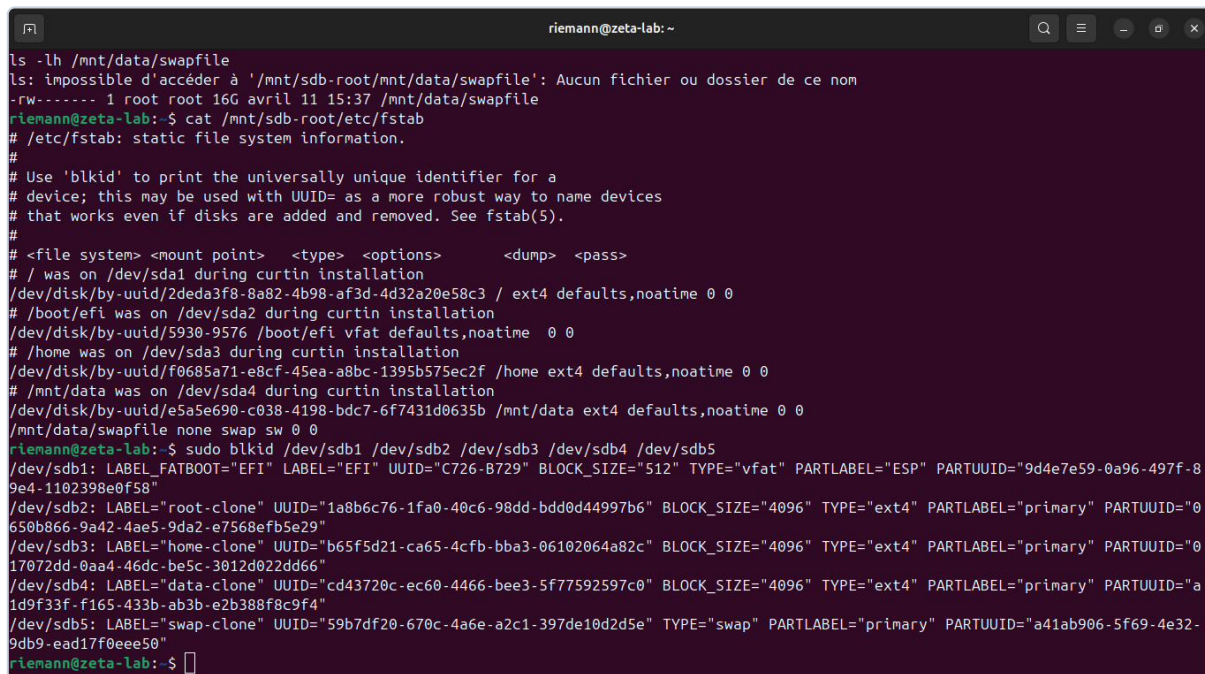
```
riemann@zeta-lab: ~  
sudo umount /mnt/sdb-root/proc  
sudo umount /mnt/sdb-root/sys  
sudo umount /mnt/sdb-root/run  
sudo umount /mnt/sdb-root/boot/efi  
umount: /mnt/sdb-root/dev/pts: Aucun point de montage indiqué.  
umount: /mnt/sdb-root/dev: non monté.  
umount: /mnt/sdb-root/proc: non monté.  
umount: /mnt/sdb-root/sys: non monté.  
umount: /mnt/sdb-root/run: non monté.  
umount: /mnt/sdb-root/boot/efi: non monté.  
riemann@zeta-lab:~$ ls -lh "/mnt/sdb-root/mnt/data/swapfile"  
ls -lh /mnt/data/swapfile  
ls: impossible d'accéder à '/mnt/sdb-root/mnt/data/swapfile': Aucun fichier ou dossier de ce nom  
-rw----- 1 root root 16G avril 11 15:37 /mnt/data/swapfile  
riemann@zeta-lab:~$ cat /mnt/sdb-root/etc/fstab  
# /etc/fstab: static file system information.  
#  
# Use 'blkid' to print the universally unique identifier for a  
# device; this may be used with UUID= as a more robust way to name devices  
# that works even if disks are added and removed. See fstab(5).  
#  
# <file system> <mount point> <type> <options> <dump> <pass>  
# / was on /dev/sda1 during curtin installation  
/dev/disk/by-uuid/2deda3f8-8a82-4b98-af3d-4d32a20e58c3 / ext4 defaults,noatime 0 0  
# /boot/efi was on /dev/sda2 during curtin installation  
/dev/disk/by-uuid/5930-9576 /boot/efi vfat defaults,noatime 0 0  
# /home was on /dev/sda3 during curtin installation  
/dev/disk/by-uuid/f0685a71-e8cf-45ea-a8bc-1395b575ec2f /home ext4 defaults,noatime 0 0  
# /mnt/data was on /dev/sda4 during curtin installation  
/dev/disk/by-uuid/e5a5e690-c038-4198-bdc7-6f7431d0635b /mnt/data ext4 defaults,noatime 0 0  
/mnt/data/swapfile none swap sw 0 0  
riemann@zeta-lab:~$
```

Capture — relevé des nouveaux UUID via `blkid`. Les 5 partitions du Toshiba sont listées avec leurs labels (`root-clone`, `home-clone`, `data-clone`, `swap-clone`) et UUID propres, prêts à remplacer les anciennes références dans le `fstab` du clone.

```

sudo sed -i 's/ANCIEN-UUID-ROOT/NOUVEAU-UUID-ROOT/' /mnt/clone-root/etc/fstab
sudo sed -i 's/ANCIEN-UUID-EFI/NOUVEAU-UUID-EFI/' /mnt/clone-root/etc/fstab
sudo sed -i 's/ANCIEN-UUID-HOME/NOUVEAU-UUID-HOME/' /mnt/clone-root/etc/fstab
sudo sed -i 's/ANCIEN-UUID-DATA/NOUVEAU-UUID-DATA/' /mnt/clone-root/etc/fstab
sudo sed -i 's#/mnt/data/swapfile none swap sw 0 0#UUID=NOUVEAU-UUID-SWAP none swap sw 0
0#' \
/mnt/clone-root/etc/fstab

```



```

riemann@zeta-lab: ~
ls -lh /mnt/data/swapfile
ls: impossible d'accéder à '/mnt/sdb-root/mnt/data/swapfile': Aucun fichier ou dossier de ce nom
-rw-r----- 1 root root 16G avril 11 15:37 /mnt/data/swapfile
riemann@zeta-lab:~$ cat /mnt/sdb-root/etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options> <dump> <pass>
# / was on /dev/sda1 during curtin installation
/dev/disk/by-uuid/2deda3f8-8a82-4b98-af3d-4d32a20e58c3 / ext4 defaults,noatime 0 0
# /boot/efi was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/5930-9576 /boot/efi vfat defaults,noatime 0 0
# /home was on /dev/sda3 during curtin installation
/dev/disk/by-uuid/f0685a71-e8cf-45ea-a8bc-1395b575ec2f /home ext4 defaults,noatime 0 0
# /mnt/data was on /dev/sda4 during curtin installation
/dev/disk/by-uuid/e5a5e690-c038-4198-bdc7-6f7431d0635b /mnt/data ext4 defaults,noatime 0 0
/mnt/data/swapfile none swap sw 0 0
riemann@zeta-lab:~$ sudo blkid /dev/sdb1 /dev/sdb2 /dev/sdb3 /dev/sdb4 /dev/sdb5
/dev/sdb1: LABEL_FATBOOT="EFI" LABEL="EFI" UUID="C726-B729" BLOCK_SIZE="512" TYPE="vfat" PARTLABEL="ESP" PARTUUID="9d4e7e59-0a96-497f-89e4-1102398e0f58"
/dev/sdb2: LABEL="root-clone" UUID="1a8b6c76-1fa0-40c6-98dd-bdd0d44997b6" BLOCK_SIZE="4096" TYPE="ext4" PARTLABEL="primary" PARTUUID="0650b866-9a42-4ae5-9da2-e7568efb5e29"
/dev/sdb3: LABEL="home-clone" UUID="b65f5d21-ca65-4cfb-bba3-06102064a82c" BLOCK_SIZE="4096" TYPE="ext4" PARTLABEL="primary" PARTUUID="017072dd-0aa4-46dc-be5c-3012d022dd66"
/dev/sdb4: LABEL="data-clone" UUID="cd43720c-ec60-4466-bee3-5f77592597c0" BLOCK_SIZE="4096" TYPE="ext4" PARTLABEL="primary" PARTUUID="a1d9f33f-f165-433b-ab3b-e2b388f8c9f4"
/dev/sdb5: LABEL="swap-clone" UUID="59b7df20-670c-4a6e-a2c1-397de10d2d5e" TYPE="swap" PARTLABEL="primary" PARTUUID="a41ab906-5f69-4e32-9db9-ead17f0eee50"
riemann@zeta-lab:~$

```

Capture — les quatre commandes `sed -i` appliquées (root, EFI, home, data) ainsi que le remplacement de la ligne `swapfile` par l'UUID de la partition swap dédiée du clone. Le `cat` final confirme un `fstab` entièrement corrigé, pointant exclusivement vers les nouveaux UUID du Toshiba.

```

sudo umount /mnt/clone-root/boot/efi /mnt/clone-root/mnt/data \
/mnt/clone-root/home /mnt/clone-root

```

```
riemann@zeta-lab: ~  
riemann@zeta-lab:~$ sudo sed -i 's/2deda3f8-8a82-4b98-af3d-4d32a20e58c3/1a8b6c76-1fa0-40c6-98dd-bdd0d44997b6/' /mnt/sdb-root/etc/fstab  
sudo sed -i 's/5930-9576/C726-B729/' /mnt/sdb-root/etc/fstab  
sudo sed -i 's/f0685a71-e8cf-45ea-a8bc-1395b575ec2f/b65f5d21-ca65-4cfb-bba3-06102064a82c/' /mnt/sdb-root/etc/fstab  
sudo sed -i 's/e5a5e690-c038-4198-bdc7-6f7431d0635b/cd43720c-ec60-4466-bee3-5f77592597c0/' /mnt/sdb-root/etc/fstab  
riemann@zeta-lab:~$ cat /mnt/sdb-root/etc/fstab  
# /etc/fstab: static file system information.  
#  
# Use 'blkid' to print the universally unique identifier for a  
# device; this may be used with UUID= as a more robust way to name devices  
# that works even if disks are added and removed. See fstab(5).  
#  
# <file system> <mount point> <type> <options> <dump> <pass>  
# / was on /dev/sda1 during curtin installation  
/dev/disk/by-uuid/1a8b6c76-1fa0-40c6-98dd-bdd0d44997b6 / ext4 defaults,noatime 0 0  
# /boot/efi was on /dev/sda2 during curtin installation  
/dev/disk/by-uuid/C726-B729 /boot/efi vfat defaults,noatime 0 0  
# /home was on /dev/sda3 during curtin installation  
/dev/disk/by-uuid/b65f5d21-ca65-4cfb-bba3-06102064a82c /home ext4 defaults,noatime 0 0  
# /mnt/data was on /dev/sda4 during curtin installation  
/dev/disk/by-uuid/cd43720c-ec60-4466-bee3-5f77592597c0 /mnt/data ext4 defaults,noatime 0 0  
UUID=59b7df20-670c-4a6e-a2c1-397de10d2d5e none swap sw 0 0  
riemann@zeta-lab:~$
```

Capture — démontage final propre des points de montage du clone, une fois le `fstab` corrigé et vérifié.

5.6 — Résultat

✓ **SUCCÈS** — Le clone Ubuntu sur le Toshiba a démarré correctement dès le premier test de boot, sans erreur supplémentaire.

6. Phase D — Dépannage boot Kali sur Samsung

Une fois les données Kali copiées sur le Samsung (Phase B), le disque a été repartitionné et rendu bootable de façon autonome. Cette phase a rencontré le plus grand nombre de difficultés du projet.

6.1 — Correction du fstab (même type d'erreur qu'en Phase C)

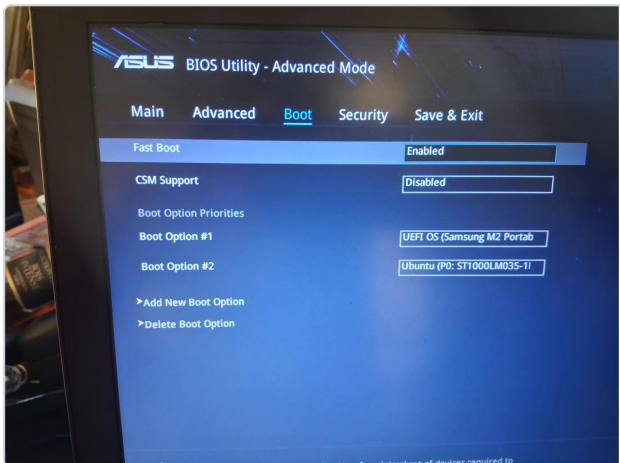
Élément	Ancien UUID (Toshiba/Kali d'origine)	Nouveau UUID (Samsung)
Partition racine (/)	2657601b-dd77-4569-b535-bb2b788db71f	eea06119-68d1-4a84-811b-c2e212b8b348
Partition EFI	2257-5D4C	7F66-0C9E
Swap	Ligne supprimée (pas de swap sur le clone)	

Le fichier `/boot/grub/grub.cfg` a été vérifié et trouvé déjà correct, référençant directement le bon UUID de `sdb2` (label `kali-clone`).

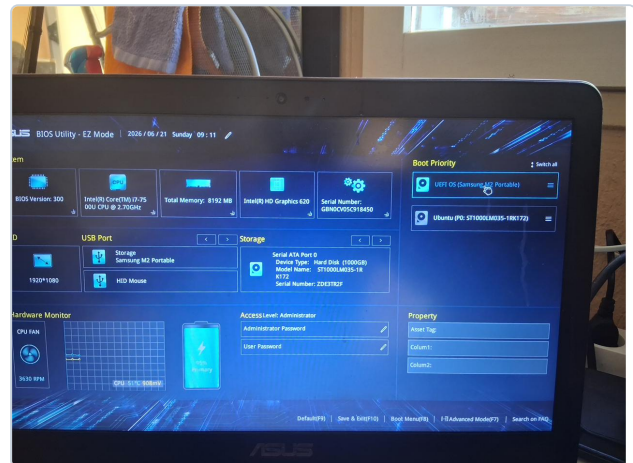
6.2 — Problème de détection BIOS (carte mère Asus)

⚠ Problème — Le BIOS Asus ne démarre pas l'entrée EFI Samsung malgré Boot Priority #1

Malgré une Boot Priority correctement réglée sur le Samsung en position 1, le firmware redémarrait systématiquement sur le disque interne Ubuntu (`sda`). Diagnostic : comportement connu sur certaines cartes Asus où les entrées NVRAM EFI personnalisées (`EFI/ubuntu/grubx64.efi`) ne sont pas fiablement relancées, contrairement au chemin de secours standard.



BIOS — onglet Boot, Advanced Mode. *Boot Option #1* = *UEFI OS (Samsung M2 Portab...)*, *Boot Option #2* = *Ubuntu*. La priorité est correctement réglée, mais le démarrage échoue encore à ce stade.

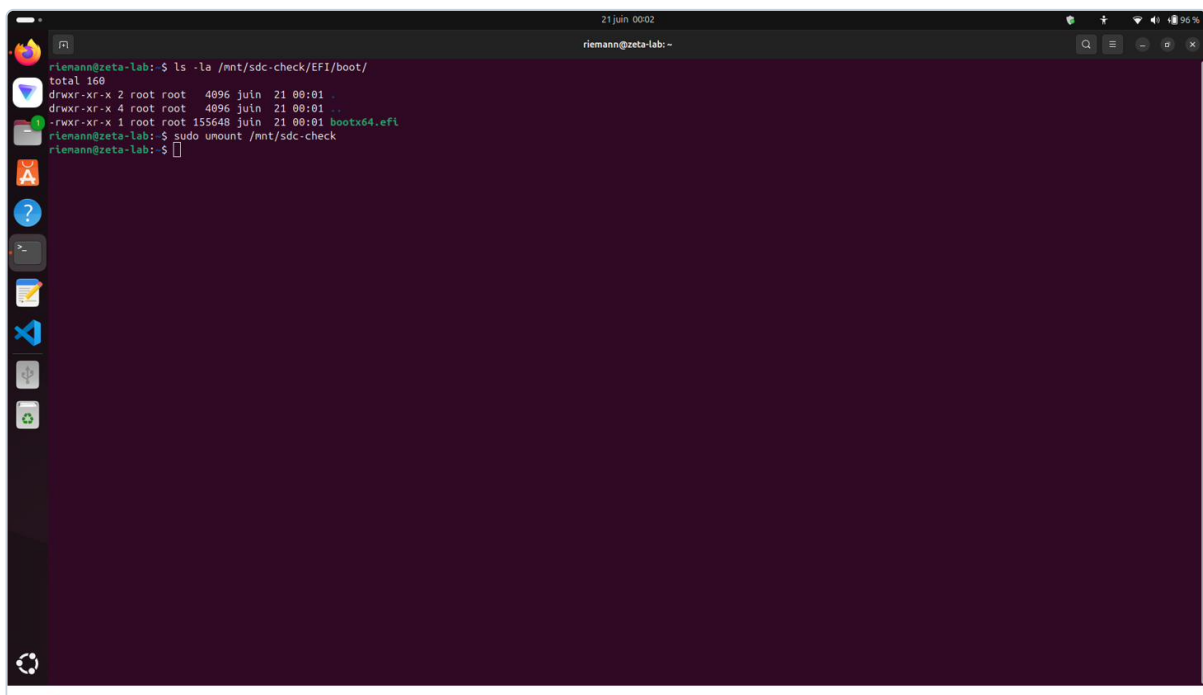


BIOS — EZ Mode, après correction. *Le Samsung M2 Portable* apparaît bien en première position de *Boot Priority*, confirmé visuellement en haut à droite de l'écran.

✓ Correction appliquée — Création d'un binaire EFI de secours (fallback)

Création du chemin EFI standard reconnu par défaut par la plupart des firmwares, en copiant le binaire GRUB existant :

```
sudo mkdir -p /boot/efi/EFI/boot
sudo cp /boot/efi/EFI/ubuntu/grubx64.efi /boot/efi/EFI/boot/bootx64.efi
```



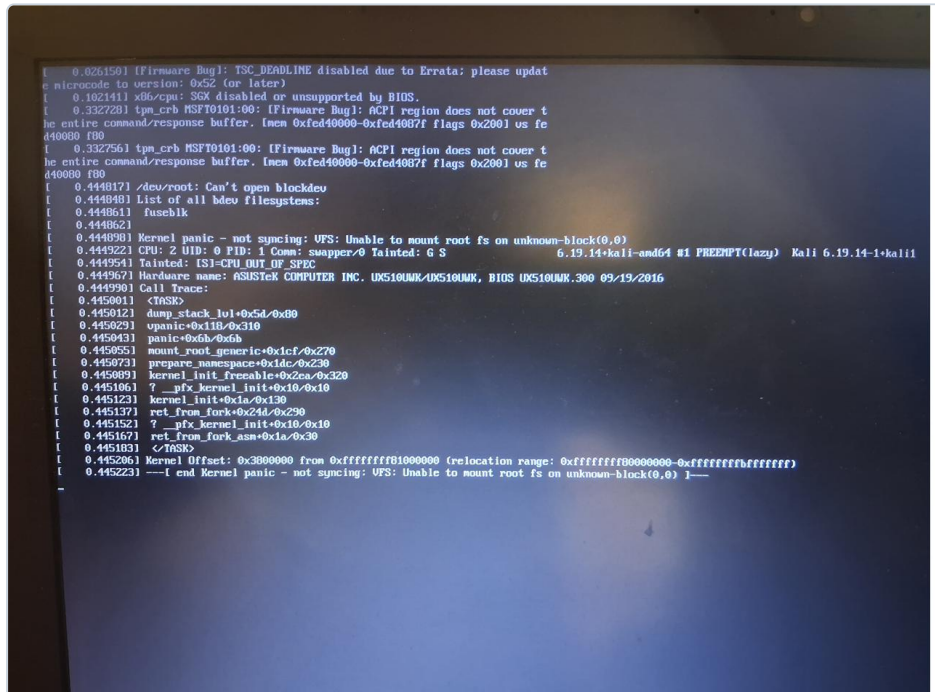
Capture — vérification du fallback créé. *ls -la /mnt/sdc-check/EFI/boot/* confirme la présence de *bootx64.efi* (155 648 octets), copie du binaire GRUB d'origine.

Confirmé via la capture *F2_fix_samsung_fisr_ok* ci-dessus : le démarrage via cette entrée de secours a permis de relancer correctement GRUB.

6.3 — Kernel panic au démarrage (montage de la racine impossible)

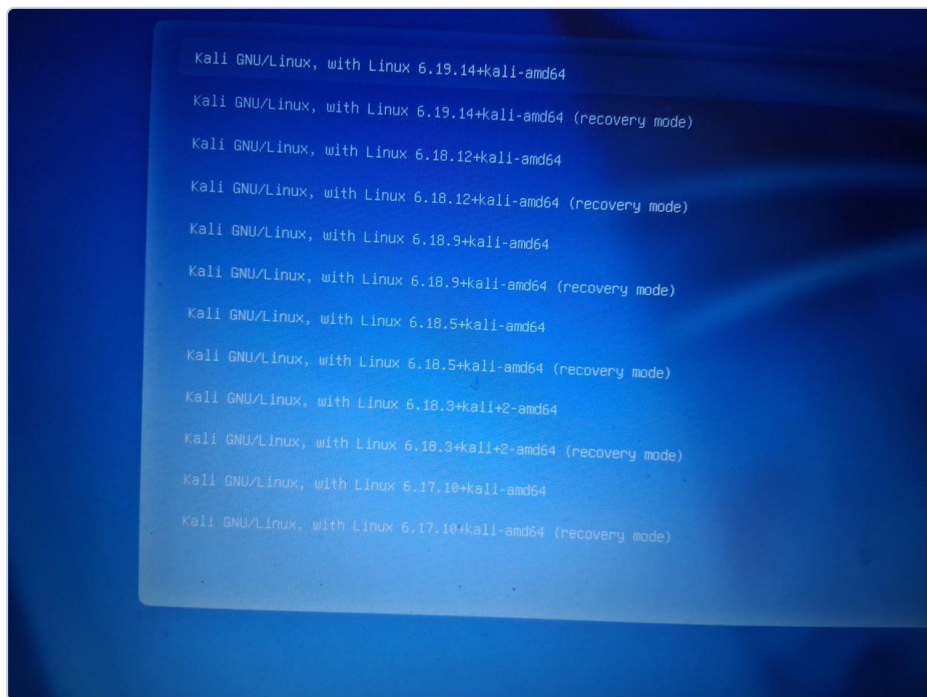
⚠ Problème — Kernel panic : « VFS: Unable to mount root fs on unknown-block(0,0) »

Une fois GRUB lancé correctement, le noyau Kali plantait avant même de monter le système de fichiers racine.



Capture — trace complète du kernel panic. [0.448898] Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) , noyau 6.19.14+kali-amd64 , carte mère identifiée dans la trace : ASUSTeK COMPUTER INC. UX510UWK/UX510UWK, BIOS UX510UWK.300 09/19/2016 .

Diagnostic effectué par chroot depuis l'Ubuntu de la machine (sda) : l'initramfs des 6 noyaux Kali installés (versions 6.17.10 à 6.19.14+kali-amd64) ne contenait aucun module de pilote de stockage USB (usb_storage, uas, xhci_hcd, xhci_pci). Le noyau ne pouvait donc physiquement pas accéder au disque racine, celui-ci étant connecté en USB et non en SATA/NVMe interne.



Menu GRUB — Advanced options for Kali GNU/Linux. Les 6 noyaux installés sont visibles, chacun avec son mode normal et son mode recovery : 6.19.14, 6.18.12, 6.18.9, 6.18.5, 6.18.3+2, 6.17.10 (+kali-amd64).

✓ Correction appliquée — Ajout des modules USB à l'initramfs

```
echo "usb_storage" | sudo tee -a /etc/initramfs-tools/modules
echo "uas" | sudo tee -a /etc/initramfs-tools/modules
echo "xhci_hcd" | sudo tee -a /etc/initramfs-tools/modules
echo "xhci_pci" | sudo tee -a /etc/initramfs-tools/modules

sudo update-initramfs -u -k all

for k in 6.17.10 6.18.3+2 6.18.5 6.18.9 6.18.12 6.19.14; do
    echo "=== ${k}+kali-amd64 ==="
    lsinitramfs /boot/initrd.img-${k}+kali-amd64 2>/dev/null | grep -E "usb_storage|uas|xhci"
done
```

⚠ Problème — Noyau 6.19.14 (le plus récent, par défaut) omis de la première régénération

La vérification a révélé que 5 des 6 noyaux contenaient bien les modules USB requis, mais que le noyau `6.19.14+kali-amd64` — le premier proposé par GRUB par défaut — était resté complètement vide de tout module USB après la commande `update-initramfs -u -k all`.

✓ Correction appliquée — Régénération ciblée du noyau manquant

```
sudo update-initramfs -u -k 6.19.14+kali-amd64
lsinitramfs /boot/initrd.img-6.19.14+kali-amd64 2>/dev/null | grep -E "usb_storage|uas|xhci"
```

Résultat confirmé : les modules `xhci-hcd.ko.xz`, `xhci-pci.ko.xz`, `uas.ko.xz` sont bien présents pour ce noyau après régénération ciblée — identique aux 5 autres noyaux.

i Avertissement bénin observé pendant chaque régénération

```
cryptsetup: ERROR: Couldn't resolve device UUID=d599a81f-4ba1-454b-a636-67a061015938
W: initramfs-tools configuration sets RESUME=UUID=... but no matching swap device is
available.
```

Diagnostiqué comme sans danger : il reflète simplement l'absence de partition swap sur le clone Kali (le fichier `/etc/initramfs-tools/conf.d/resume` référençait encore le swap du système d'origine). Ce point a été définitivement corrigé en Phase E (voir §7.6).

6.4 — Réinstallation de GRUB en mode amovible

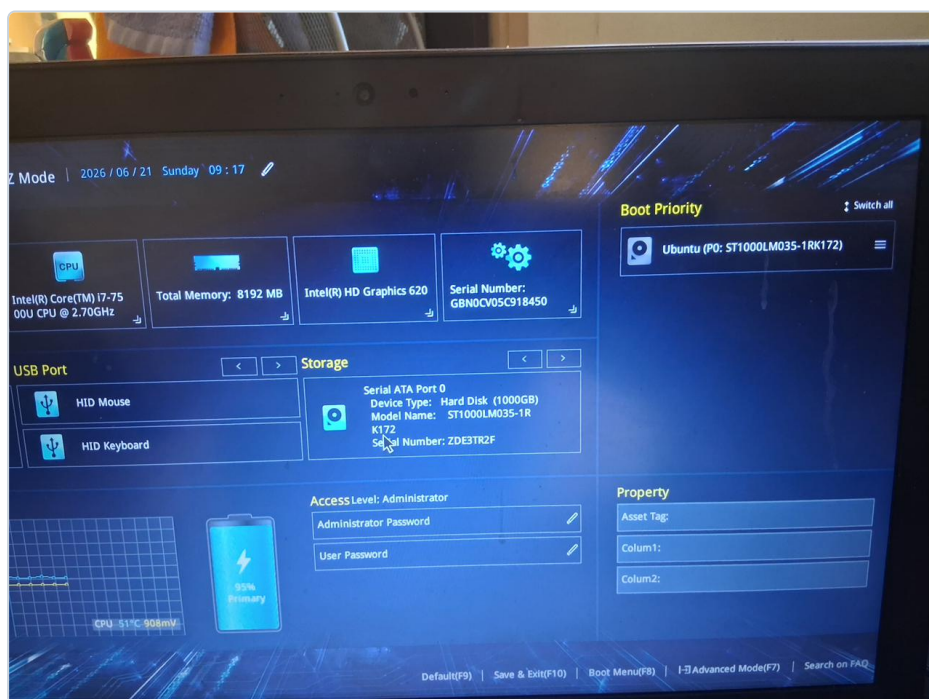
```
sudo grub-install --target=x86_64-efi --efi-directory=/boot/efi \
--bootloader-id=kali --removable /dev/sdb
sudo update-grub
```

L'option `--removable` garantit l'écriture également du chemin EFI de secours standard, renforçant la correction faite manuellement en §6.2.

6.5 — Problème de réapparition/disparition du Samsung en Boot Priority

⚠ Problème observé — Le Samsung disparaît de la liste Boot Priority après passage par UEFI Firmware Settings

Après être passé par le menu UEFI Firmware Settings depuis GRUB puis avoir sauvegardé, seule l'entrée Ubuntu (P0: ST1000LM035...) restait visible en Boot Priority — le Samsung avait disparu.



Capture — preuve de la disparition. BIOS EZ Mode, encadré « Boot Priority » à droite : seule l'entrée `Ubuntu (P0: ST1000LM035-1RK172)` est listée, le Samsung n'apparaît plus du tout.

6.6 — Diagnostic complémentaire (instabilité des lettres de disque)

[illegible]

Tentative de montage sur la mauvaise lettre. `sudo mount /dev/sdc1 /mnt/sdc-check` échoue (« le périphérique spécial /dev/sdc1 n'existe pas ») — le disque s'appelle désormais `sdb`, pas `sdc`, suite à un rebranchement.

```

riemann@zeta-lab:~$ lsblk -f

```

NAME	FSTYPE	FSTYPE LABEL	UUID	FSAAVAIL	FSUSEK	MOUNTPOINTS
loop0	squashfs	4.0		0	100%	/snap/canonical-livepatch/393
loop1	squashfs	4.0		0	100%	/snap/bare/5
loop2	squashfs	4.0		0	100%	/snap/canonical-livepatch/406
loop3	squashfs	4.0		0	100%	/snap/code/243
loop4	squashfs	4.0		0	100%	/snap/code/247
loop5	squashfs	4.0		0	100%	/snap/core20/2769
loop6	squashfs	4.0		0	100%	/snap/core20/2866
loop7	squashfs	4.0		0	100%	/snap/core22/2292
loop8	squashfs	4.0		0	100%	/snap/core22/2411
loop9	squashfs	4.0		0	100%	/snap/core24/1587
loop10	squashfs	4.0		0	100%	/snap/core24/1643
loop11	squashfs	4.0		0	100%	/snap/firefox/8462
loop12	squashfs	4.0		0	100%	/snap/firefox/8584
loop13	squashfs	4.0		0	100%	/snap/firmware-updater/226
loop14	squashfs	4.0		0	100%	/snap/firmware-updater/224
loop15	squashfs	4.0		0	100%	/snap/glow/99
loop16	squashfs	4.0		0	100%	/snap/gnome-42-2204/247
loop17	squashfs	4.0		0	100%	/snap/gnome-46-2404/153
loop18	squashfs	4.0		0	100%	/snap/mesa-2404/1165
loop19	squashfs	4.0		0	100%	/snap/gtk-common-themes/1535
loop20	squashfs	4.0		0	100%	/snap/snap-store/1338
loop21	squashfs	4.0		0	100%	/snap/snap-store/1367
loop22	squashfs	4.0		0	100%	/snap/snapd/26382
loop23	squashfs	4.0		0	100%	/snap/snapd/26865
loop24	squashfs	4.0		0	100%	/snap/snapd-desktop-integration/387
loop25	squashfs	4.0		0	100%	/snap/snapd-desktop-integration/361
sda						
sda1 ext4	1.0		2deda3f8-8a82-4b98-af3d-4d32a20e58c3	32,8G	50%	/
sda2 vfat	FAT32		5930-9576	1G	1%	/boot/efi
sda3 ext4	1.0		f8685a71-e8cf-45ea-a8bc-1395b575ec2f	59,5G	62%	/home
sda4 ext4	1.0		e5a5e600-c03b-4190-bdc7-6f7431d0835b	589,7G	4%	/mnt/data
sda5 swap	1		1e353ef5-4d66-45a9-9ca2-b298072dfd38			
sdb						
sdb1 vfat	FAT32	EFI	7f66-0c9e			
sdb2 ext4	1.0		kali.clone eaa06119-68d1-4a84-811b-c2e212b8b348	221G	46%	/media/riemann/kali-clone

```

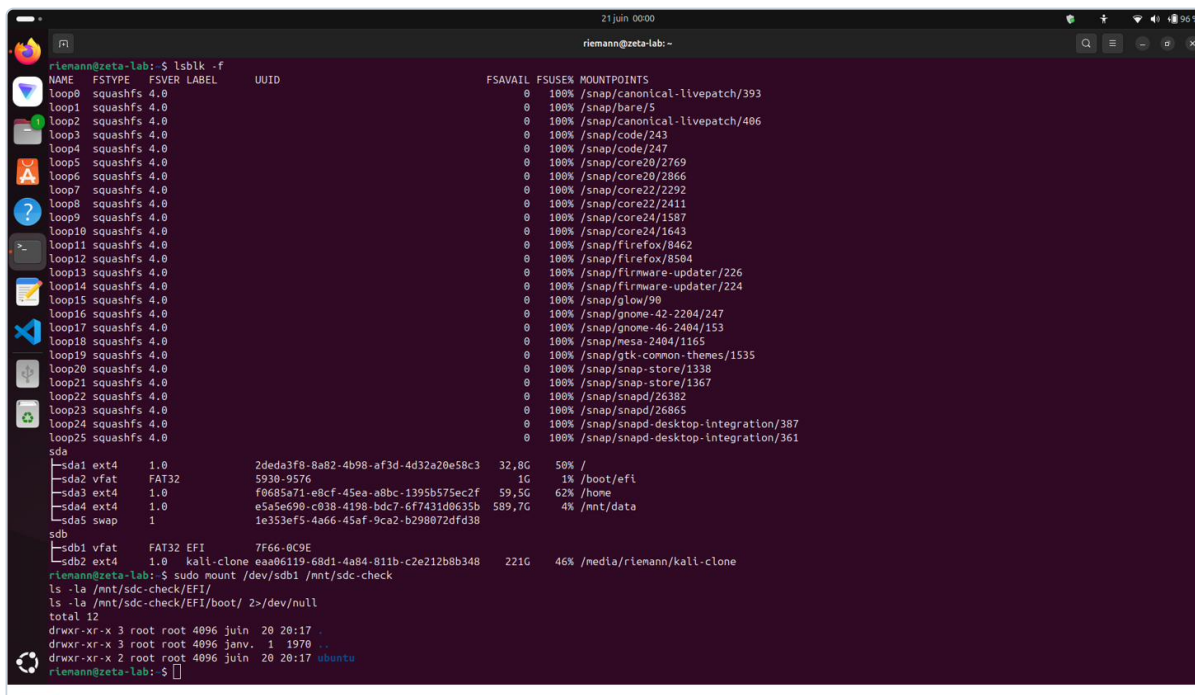
riemann@zeta-lab:~$

```

Riemann Lab — Projet Zêta — Clonage multi-disques Page 26 / 40

6.7 — Résultat final de la Phase D

✓ **SUCCÈS** — Démarrage confirmé et validé du clone Kali en standalone sur le disque Samsung, après application cumulée de : fallback EFI (§6.2), correction fstab (§6.1), ajout des modules USB à l'initramfs pour tous les noyaux (§6.3), et réinstallation GRUB en mode **--removable** (§6.4).



```
riemann@zeta-lab:~$ lsblk -f
NAME        FSTYPE LABEL        UUID                               FSAVAIL FSUSE% MOUNTPOINTS
loop0       squashfs 4.0
loop1       squashfs 4.0
loop2       squashfs 4.0
loop3       squashfs 4.0
loop4       squashfs 4.0
loop5       squashfs 4.0
loop6       squashfs 4.0
loop7       squashfs 4.0
loop8       squashfs 4.0
loop9       squashfs 4.0
loop10      squashfs 4.0
loop11      squashfs 4.0
loop12      squashfs 4.0
loop13      squashfs 4.0
loop14      squashfs 4.0
loop15      squashfs 4.0
loop16      squashfs 4.0
loop17      squashfs 4.0
loop18      squashfs 4.0
loop19      squashfs 4.0
loop20      squashfs 4.0
loop21      squashfs 4.0
loop22      squashfs 4.0
loop23      squashfs 4.0
loop24      squashfs 4.0
loop25      squashfs 4.0
sda
├─sda1 ext4    1.0      2deda3f8-8a82-4b98-af3d-4d32a20e58c3 32,8G  50% /
├─sda2 vfat    FAT32    5930-9576                               1G      1% /boot/efi
├─sda3 ext4    1.0      f0605a71-e8cf-45ea-a8bc-1395b575ec2f 59,5G  62% /home
├─sda4 ext4    1.0      e5a5e690-c938-4198-bdc7-6f7431d0635b 589,7G  4% /mnt/data
└─sda5 swap    1        1e353ef5-4a66-45af-9ca2-b298072df438
sdb
├─sdb1 vfat    FAT32    7F66-0C9E                               221G    46% /media/riemann/kali-clone
└─sdb2 ext4    1.0      eaa06119-68d1-4a84-811b-c2e212b8b348
riemann@zeta-lab:~$ sudo mount /dev/sdb1 /mnt/sdc-check
riemann@zeta-lab:~$ ls -la /mnt/sdc-check/EFI/
total 12
drwxr-xr-x 3 root root 4096 juin 20 20:17 .
drwxr-xr-x 3 root root 4096 janv. 1 1970 ..
drwxr-xr-x 2 root root 4096 juin 20 20:17 ubuntu
riemann@zeta-lab:~$
```

Capture — confirmation finale (21 juin, 00h00). `lsblk -f` confirme `sdb` = label `kali-clone`, montée et exploitable de façon autonome.

7. Phase E — Première mise à jour système du clone Kali

Une fois le boot du clone Kali confirmé de façon autonome (sans dépendre du disque Ubuntu pour le chroot), une mise à jour standard du système a été lancée pour vérifier la stabilité complète du clone.

7.1 — Erreur de syntaxe sur dpkg (tiret long copié-collé)

⚠ Problème — Option « -♦ inconnue »

```
$ sudo dpkg --configure -a
dpkg: erreur: option -♦ inconnue
```

Cause : un copier-coller depuis un éditeur de texte a substitué le double tiret court (`--`) par un tiret long unique (`-`, caractère Unicode U+2013), non reconnu par `dpkg`.

✓ Correction appliquée

Re-saisie manuelle de la commande au clavier, garantissant l'usage de deux tirets courts ASCII :

```
sudo dpkg --configure -a
```

7.2 — Erreur de permissions sur apt (sudo omis)

⚠ Problème — Verrou apt inaccessible

```
$ apt update && apt upgrade
Erreur : Impossible d'ouvrir le fichier verrou /var/lib/apt/lists/lock
- open (13: Permission non accordée)
```

Cause : commande lancée sans `sudo`, l'utilisateur standard n'ayant pas les droits d'écriture sur `/var/lib/apt/`.

✓ Correction appliquée

```
sudo apt update && sudo apt upgrade -y
```

7.3 — Conflit de configuration sur /etc/sudoers

i Décision prise pendant l'upgrade

`dpkg` a signalé que `/etc/sudoers` avait été modifié localement (probablement par un script d'installation pendant le clonage) alors qu'une nouvelle version officielle du paquet existait. Conformément au principe de prudence (ne jamais remplacer à l'aveugle un fichier de contrôle des droits sudo fonctionnel), l'option « **N** » (conserver la version actuelle) a été choisie — c'était également l'option par défaut suggérée par `dpkg`.

7.4 — Messages D-Bus en rouge pendant l'upgrade (faux positif)

i Observation — Lignes en rouge pendant l'installation des paquets

```
Failed to get properties: Noeud final de transport n'est pas connecté
Failed to connect to system scope bus via local transport: Connexion refusée
```

Diagnostic — sans gravité, aucune action requise. Ces messages proviennent de tentatives de `systemctl` / `polkit` de notifier des changements de service via D-Bus, dans un contexte où certains sockets système n'étaient pas encore pleinement disponibles. `dpkg` a continué normalement l'installation des paquets suivants (`polkitd` , `fontconfig` , `kali-menu` , etc.) sans interruption ni corruption.

7.5 — Échec de redémarrage du service VirtualBox (faux positif)

i Observation — Lignes en rouge liées à VirtualBox

```
Unit virtualbox.service could not be found.
Failed to restart virtualbox.service: Unit virtualbox.service not found.
invoke-rc.d: initscript virtualbox, action "restart" failed.
```

Diagnostic — sans gravité, modules kernel correctement construits. Le paquet `virtualbox-dkms` a bien reconstruit et signé les modules noyau (`vboxdrv` , `vboxnetadp` , `vboxnetflt`) pour chaque version de noyau installée (étapes *Building module(s)...* *done* confirmées avant l'erreur). Seule la tentative finale de redémarrage du service `virtualbox.service` a échoué, ce service n'étant pas configuré comme actif sur ce système. Sans incidence : les modules seront chargés au prochain redémarrage, ou immédiatement via :

```
sudo modprobe vboxdrv
```

7.6 — Warning UUID swap résiduel pendant la régénération d'initramfs post-upgrade

⚠ Problème — RESUME=UUID pointant vers un swap inexistant sur le clone

```
cryptsetup: ERROR: Couldn't resolve device UUID=d599a81f-4ba1-454b-a636-67a061015938
W: initramfs-tools configuration sets RESUME=UUID=... but no matching swap device is
available.
```

Confirmé par inspection du disque (`lsblk -f`) : le clone Kali (label `kali-clone` , partition unique `sdb2`) ne possède aucune partition swap. Le fichier `/etc/initramfs-tools/conf.d/resume` référençait encore l'UUID du swap de la machine source d'origine du clone.

✓ Correction appliquée — Désactivation explicite du resume

```
cat /etc/initramfs-tools/conf.d/resume
→ RESUME=UUID=d599a81f-4ba1-454b-a636-67a061015938

sudo sed -i 's/^RESUME=.*RESUME=none/' /etc/initramfs-tools/conf.d/resume
cat /etc/initramfs-tools/conf.d/resume
```

```
→ RESUME=none
```

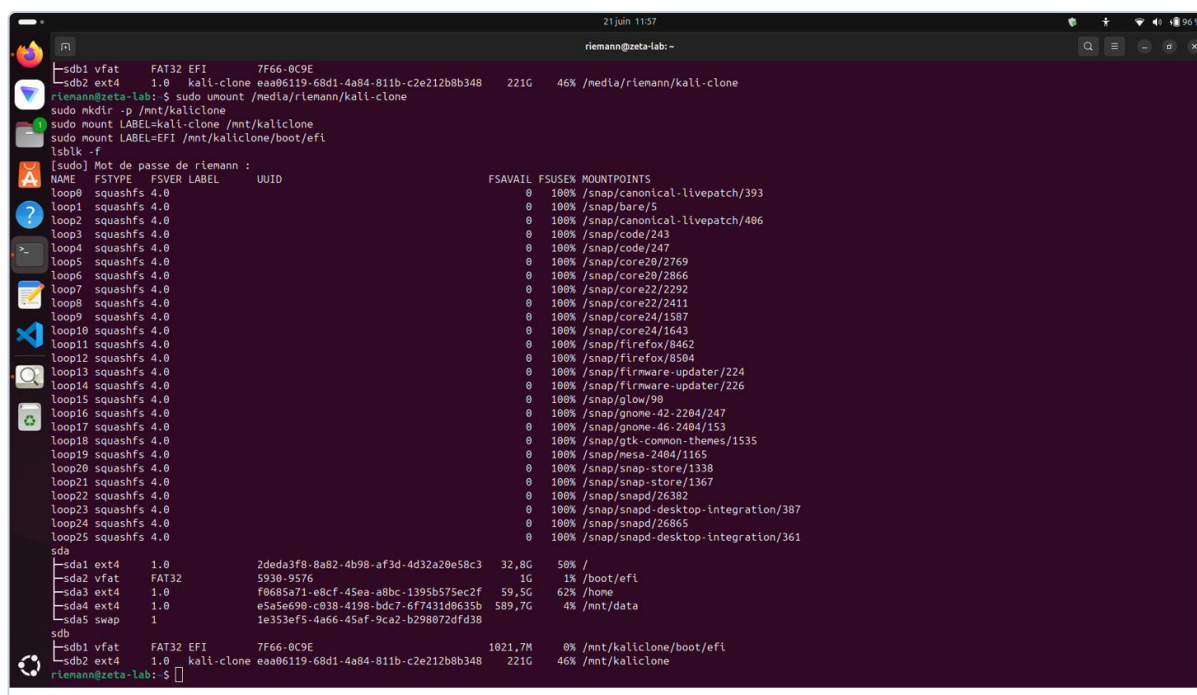
```
sudo update-initramfs -u -k all
```

Régénération de l'initramfs pour tous les noyaux afin que le changement soit pris en compte au prochain démarrage. Un redémarrage à froid ultérieur a permis de confirmer la disparition définitive du warning swap, validant le clone Kali comme entièrement stable et autonome.

7.7 — Vérifications complémentaires (re-diagnostic du 21 juin matin)

Une nouvelle session de vérification a été menée le lendemain matin pour confirmer la stabilité du clone, cette fois en montant les partitions Kali par **label** plutôt que par lettre de périphérique, conformément à la règle retenue au §2.

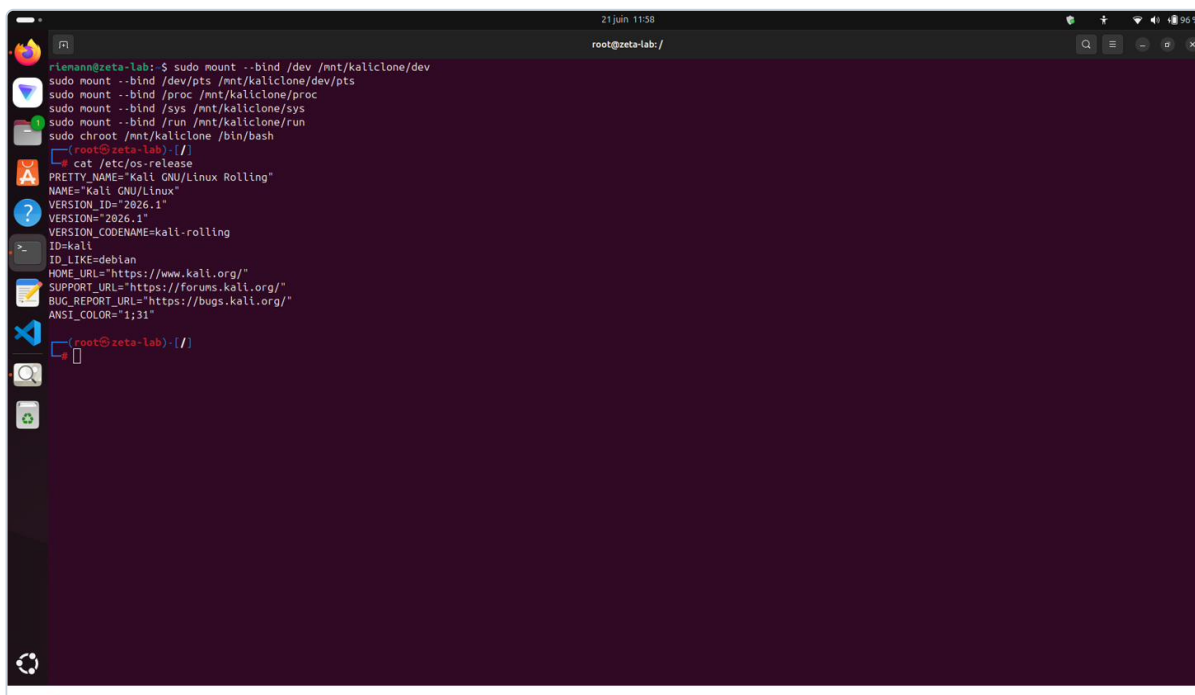
```
sudo mkdir -p /mnt/kaliclone
sudo mount LABEL=kali-clone /mnt/kaliclone
sudo mount LABEL=EFI /mnt/kaliclone/boot/efi
```



```
riemann@zeta-lab: ~
21 juin 11:57
lsblk -f
NAME FSTYPE FSVER LABEL UUID FSAVAIL FSUSE% MOUNTPOINTS
loop0 squashfs 4.0 0 100% /snap/canonical-livepatch/393
loop1 squashfs 4.0 0 100% /snap/bare/5
loop2 squashfs 4.0 0 100% /snap/canonical-livepatch/486
loop3 squashfs 4.0 0 100% /snap/code/243
loop4 squashfs 4.0 0 100% /snap/code/247
loop5 squashfs 4.0 0 100% /snap/core20/2769
loop6 squashfs 4.0 0 100% /snap/core20/2866
loop7 squashfs 4.0 0 100% /snap/core22/2292
loop8 squashfs 4.0 0 100% /snap/core22/2411
loop9 squashfs 4.0 0 100% /snap/core24/1587
loop10 squashfs 4.0 0 100% /snap/core24/1643
loop11 squashfs 4.0 0 100% /snap/firefox/8462
loop12 squashfs 4.0 0 100% /snap/firefox/8504
loop13 squashfs 4.0 0 100% /snap/firmware-updater/224
loop14 squashfs 4.0 0 100% /snap/firmware-updater/226
loop15 squashfs 4.0 0 100% /snap/glow/90
loop16 squashfs 4.0 0 100% /snap/gnome-42-2204/247
loop17 squashfs 4.0 0 100% /snap/gnome-46-2404/153
loop18 squashfs 4.0 0 100% /snap/gtk-common-themes/1535
loop19 squashfs 4.0 0 100% /snap/nasa-2404/1165
loop20 squashfs 4.0 0 100% /snap/snap-store/1338
loop21 squashfs 4.0 0 100% /snap/snap-store/1367
loop22 squashfs 4.0 0 100% /snap/snapd/26382
loop23 squashfs 4.0 0 100% /snap/snapd-desktop-integration/387
loop24 squashfs 4.0 0 100% /snap/snapd/26865
loop25 squashfs 4.0 0 100% /snap/snapd-desktop-integration/361
sda
sda1 ext4 1.0 2deda3f8-8a82-4b98-af3d-4d32a20e58c3 32,8G 50% /
sda2 vfat FAT32 5930-9576 1G 1% /boot/efi
sda3 ext4 1.0 f0685a71-e8cf-45ea-a8bc-1395b575ec2f 59,5G 62% /home
sda4 ext4 1.0 e5a5e698-c838-4198-bdc7-6f7431d8635b 589,7G 4% /mnt/data
sda5 swap 1 1e353ef5-4a66-45af-9ca2-b298072df38
sdb
sdb1 vfat FAT32 EFI 7f66-0c9e 1021,7M 0% /mnt/kaliclone/boot/efi
sdb2 ext4 1.0 kali-clone eaa06119-68d1-4a84-811b-c2e212b8b348 221G 46% /mnt/kaliclone
riemann@zeta-lab: $
```

Capture — montage par label. `mount LABEL=kali-clone` et `mount LABEL=EFI` exécutés avec succès ; le `lsblk -f` qui suit confirme les points de montage corrects sous `/mnt/kaliclone`, indépendamment de la lettre `/dev/sdX` attribuée ce jour-là.

```
sudo mount --bind /dev /mnt/kaliclone/dev
sudo mount --bind /dev/pts /mnt/kaliclone/dev/pts
sudo mount --bind /proc /mnt/kaliclone/proc
sudo mount --bind /sys /mnt/kaliclone/sys
sudo mount --bind /run /mnt/kaliclone/run
sudo chroot /mnt/kaliclone /bin/bash
cat /etc/os-release
```



```
riemann@zeta-lab:~$ sudo mount --bind /dev /mnt/kaliclone/dev
sudo mount --bind /dev/pts /mnt/kaliclone/dev/pts
sudo mount --bind /proc /mnt/kaliclone/proc
sudo mount --bind /sys /mnt/kaliclone/sys
sudo mount --bind /run /mnt/kaliclone/run
sudo chroot /mnt/kaliclone /bin/bash
(root@zeta-lab)-[/]
# cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2026.1"
VERSION="2026.1"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"
(root@zeta-lab)-[/]
```

Capture — confirmation finale en chroot. `cat /etc/os-release` retourne `PRETTY_NAME="Kali GNU/Linux Rolling"`, `VERSION_ID="2026.1"` — le clone est bien un système Kali Rolling 2026.1 complet et cohérent.

i Avertissement bénin observé en chroot

```
sudo: impossible de résoudre l'hôte zeta-lab : Nom ou service inconnu
```

Ce message est apparu lors d'une commande `sudo` exécutée à l'intérieur du chroot du clone Kali. Cause : le fichier `/etc/hosts` du système cloné ne contient pas l'entrée locale `zeta-lab` propre à la machine hôte actuelle (résidu du système Kali d'origine). Sans incidence sur le fonctionnement réel du clone une fois démarré nativement (hors chroot) : `sudo` continue de fonctionner correctement malgré l'avertissement, celui-ci n'étant qu'un échec de résolution de nom non bloquant.

```

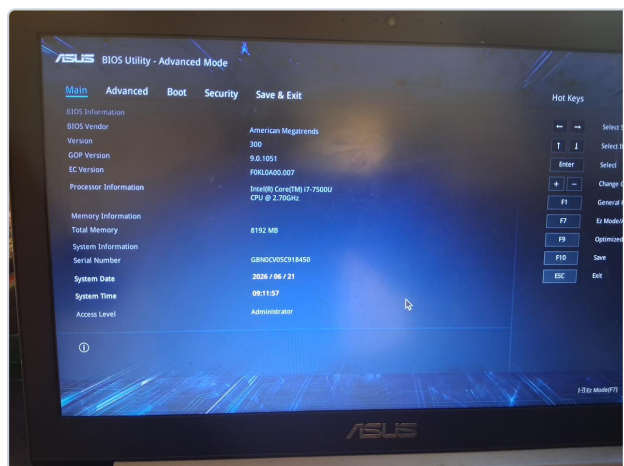
21 juin 12:16
riemann@zeta-lab:~$ sudo: impossible de résoudre l'hôte zeta-lab: Non ou service inconnu
umount: /mnt/kaliclone/boot/efi: no mount point specified.
sudo: impossible de résoudre l'hôte zeta-lab: Non ou service inconnu
umount: /mnt/kaliclone: no mount point specified.
NAME      FSTYPE    FSVER LABEL      UUID                      FSAVAIL FSUSE% MOUNTPOINTS
loop0     squashfs  4.0
loop1     squashfs  4.0
loop2     squashfs  4.0
loop3     squashfs  4.0
loop4     squashfs  4.0
loop5     squashfs  4.0
loop6     squashfs  4.0
loop7     squashfs  4.0
loop8     squashfs  4.0
loop9     squashfs  4.0
loop10    squashfs  4.0
loop11    squashfs  4.0
loop12    squashfs  4.0
loop13    squashfs  4.0
loop14    squashfs  4.0
loop15    squashfs  4.0
loop16    squashfs  4.0
loop17    squashfs  4.0
loop18    squashfs  4.0
loop19    squashfs  4.0
loop20    squashfs  4.0
loop21    squashfs  4.0
loop22    squashfs  4.0
loop23    squashfs  4.0
loop24    squashfs  4.0
loop25    squashfs  4.0
sda
├─sda1 ext4      1.0          2deda3f8-8a82-4b98-af3d-4d32a20e58c3 5930,9576
├─sda2 vfat     FAT32       5930-9576
├─sda3 ext4      1.0          f0685a71-e8cf-45ea-a8bc-1395b575ec2f 5930,9576
├─sda4 ext4      1.0          e5a5e698-c938-4198-bdc7-6f7431d8635b 5930,9576
└─sda5 swap        1           1e353ef5-4a66-45af-9ca2-b298072dfd38
sdb
├─sdb1 vfat     FAT32       7F66-0C9E 1021,7M  8% /boot/efi
└─sdb2 ext4      1.0         kalil-clone eaa06119-68d1-4a84-811b-c2e212b8b348 220,8G  47% /
riemann@zeta-lab:~$ lsblk -f
riemann@zeta-lab:~$ exit
riemann@zeta-lab:~$

```

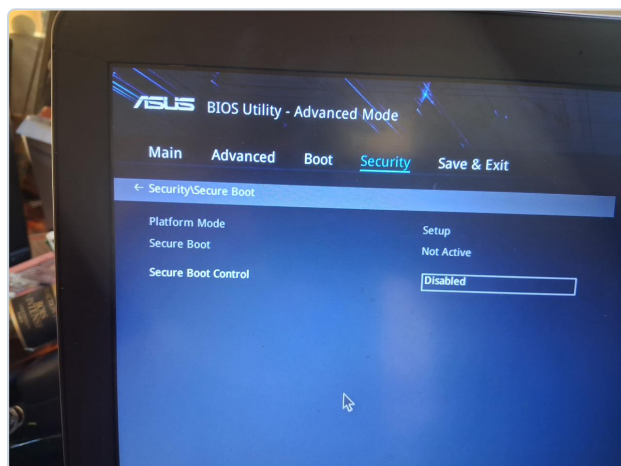
Capture — avertissement de résolution de nom en chroot et démontage complet en sortie (`umount` de tous les points liés), suivi d'un `lsblk -f` final propre confirmant l'absence de montages résiduels.

Contexte matériel — plateforme Asus de référence

L'ensemble des opérations BIOS/UEFI de ce projet a été réalisé sur la même plateforme Asus (BIOS Utility, version 300), dont les écrans de référence (informations système, Secure Boot) ont été capturés en début de projet pour documentation.



BIOS Utility — Main / Advanced Mode. Intel Core i7-7500U, 8192 MB RAM, BIOS version 300. Confirme la plateforme matérielle utilisée pour tous les tests de boot du projet.



BIOS Utility — Security / Secure Boot. Secure Boot Control : `Disabled`. Paramètre nécessaire pour permettre le démarrage de GRUB sur les disques clonés sans signature.

8. Synthèse de tous les problèmes rencontrés et corrections

#	Phase	Problème	Cause	Correction
1	Inventaire	Disque supposé 2 To en réalité 256 Go (Micron/BitLocker)	Hypothèse de départ non vérifiée	Réinventaire complet par <code>lsblk -f</code> , identification par modèle réel
2	Clonage Kali	Volume de données (721 Go) dépassant la capacité Samsung (500 Go)	Dossier <code>home/img</code> de 509 Go (dump forensique)	Exclusion validée avec l'utilisateur (<code>--exclude='home/img/'</code>)
3	Clonage Kali	Erreurs de permission lors du rsync	Commande lancée sans <code>sudo</code>	Toutes les copies système relancées avec <code>sudo + --one-file-system</code>
4	Clonage Kali	2787 lignes « error/failed/denied » dans le journal rsync	Faux positifs : noms de fichiers légitimes contenant ces mots	Filtrage ciblé <code>grep -c "^rsync:"</code> = 0 vraie erreur confirmée
5	Partitionnement Toshiba	Dépassement de capacité disque lors du calcul MiB	Confusion Go décimaux / Tio binaires	Utilisation de <code>100%</code> comme borne finale + vérif via <code>parted print</code>
6	Clonage Ubuntu	Avertissement <code>rsync: some files vanished</code> (code 24)	Fichier de cache Firefox temporaire disparu en cours de copie (système live)	Vérification ciblée du fichier concerné — confirmé bénin, aucune action requise
7	Clonage Ubuntu	fstab pointant vers les anciens UUID	UUID non régénérés après clonage	<code>sed -i</code> ciblé sur chaque UUID, vérifié via <code>blkid</code>
8	Boot Kali Samsung	BIOS Asus ne démarre pas l'entrée EFI personnalisée	Comportement connu de certaines cartes Asus (NVRAM)	Création du fallback <code>EFI/boot/bootx64.efi</code>
9	Boot Kali Samsung	Kernel panic — impossible de monter la racine	Modules USB (<code>usb_storage</code> , <code>uas</code> , <code>xhci_*</code>) absents de l'initramfs	Ajout des modules + <code>update-initramfs -u -k all</code>
10	Boot Kali Samsung	Noyau 6.19.14 (par défaut) oublié lors de la régénération groupée	Build initramfs incomplet pour ce noyau précis	Régénération ciblée <code>update-initramfs -u -k 6.19.14+kali-amd64</code>
11	Boot Kali Samsung	Samsung disparaît temporairement de Boot Priority	Rescan USB du firmware non terminé après passage UEFI Settings	Redémarrage supplémentaire — réapparition confirmée
12	Boot Kali Samsung	Tentative de montage sur <code>/dev/sdc1</code> inexistant	Lettre de périphérique changée entre deux branchements USB	Identification par <code>lsblk -f</code> et label <code>kali-clone</code>
13	Upgrade Kali			

		<code>dpkg --configure -a</code> rejeté (option inconnue)	Tiret long Unicode au lieu de deux tirets ASCII	Re-saisie manuelle de la commande
14	Upgrade Kali	Verrou apt inaccessible	<code>sudo</code> omis	<code>sudo apt update &&</code> <code>sudo apt upgrade -y</code>
15	Upgrade Kali	Conflit de version sur <code>/etc/sudoers</code>	Fichier modifié localement vs nouvelle version officielle	Conservation de la version locale (<code>N</code>)
16	Upgrade Kali	Erreurs D-Bus en rouge pendant l'upgrade	Sockets système non disponibles temporairement	Aucune — faux positif confirmé, upgrade non interrompu
17	Upgrade Kali	Échec redémarrage <code>virtualbox.service</code>	Service non configuré sur ce système, modules bien construits	Aucune — faux positif, <code>modprobe vboxdrv</code> si besoin
18	Upgrade Kali	Warning <code>RESUME=UUID</code> swap introuvable	<code>conf.d/resume</code> référéncant le swap de la machine d'origine	<code>RESUME=none</code> + régénération initramfs, confirmé par reboot à froid
19	Diagnostic chroot	<code>sudo: impossible de résoudre l'hôte zeta- lab</code>	<code>/etc/hosts</code> du clone sans l'entrée de l'hôte courant	Aucune — bénin, sans incidence hors chroot

9. État final et tableau de bord

Disque	Rôle	Partitionnement / fstab	GRUB	Test de boot	Mise à jour système
<code>sda</code> — Seagate 1 To (interne)	Système principal Ubuntu	Inchangé	Inchangé	N/A — jamais interrompu	Hors périmètre de ce rapport
Toshiba 2 To USB3	Clone Ubuntu bootable (secours)	✓ Corrigé	✓ Installé	✓ Confirmé au 1er essai	Non requise dans ce projet
Samsung 500 Go USB3	Clone Kali bootable	✓ Corrigé	✓ Installé (—removable)	✓ Confirmé après corrections	✓ Terminée — RESUME=none confirmé par reboot à froid
SSD Micron (Verbatim)	Hors projet	Non touché	Non touché	Non concerné	Non concerné

Reste à faire / maintenance continue

- Maintenance continue : tenir à jour les deux clones (Ubuntu/Toshiba et Kali/Samsung) en parallèle du système principal.
- Optionnel : exécuter `sudo modprobe vboxdrv` sur le clone Kali si l'utilisateur souhaite utiliser VirtualBox immédiatement sans redémarrer au préalable.
- Recommandé : programmer une vérification SMART périodique (`smartctl -H`) sur les deux disques externes, utilisés en clones de secours.

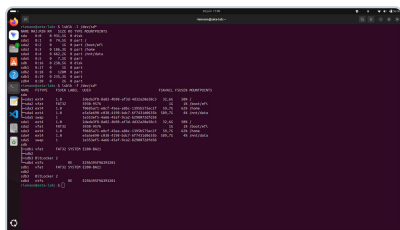
10. Leçons apprises et bonnes pratiques retenues

- **Les noms `/dev/sdX` sont instables** d'une session à l'autre selon l'ordre de branchement USB — toujours identifier les disques par label ou UUID, jamais par lettre seule, pour éviter tout risque d'opération sur le mauvais disque.
- **`rsync --one-file-system` s'arrête aux limites de montage** — pour cloner un système multi-partitions, il faut lancer une commande `rsync` distincte par partition.
- **Les calculs de tailles en MiB avec `parted` peuvent dépasser la capacité réelle** du disque à cause de la confusion entre notation décimale (Go) et binaire (Gio/Tio) — préférer les valeurs en Go confirmées par `parted print`, et utiliser `100%` pour la dernière partition.
- **Un comptage brut de mots-clés (« error », « failed ») dans un journal `rsync` volumineux produit des faux positifs massifs** — ces termes apparaissent couramment dans des noms de fichiers légitimes. Le test fiable est de filtrer les lignes préfixées `rsync:`, seules véritables émissions d'erreur de l'outil.
- **Le message `rsync: some files vanished` (code 24) est normal sur un système live** — il signale simplement qu'un fichier temporaire a disparu entre l'énumération et la copie, sans affecter l'intégrité du clone.
- **Certains firmwares Asus ne détectent pas une entrée EFI personnalisée** sans qu'un chemin de secours existe — créer systématiquement `EFI/boot/bootx64.efi` (copie de `EFI/<distro>/grubx64.efi`) pour tout disque externe bootable.
- **La correction des UUID dans `fstab` est indispensable après tout clonage** — un remplacement `sed -i` systématique des anciens UUID par les nouveaux est une étape post-clonage obligatoire, y compris pour la ligne swap si une partition swap dédiée existe sur le clone.
- **Les données volumineuses et recréables** (ex. dumps forensiques) doivent être exclues des clones — utiliser les motifs d'exclusion `rsync` pour garder l'opération réaliste en taille.
- **Pour un disque externe bootable sans swap propre, désactiver explicitement `RESUME` (`RESUME=none`)** plutôt que de laisser un UUID de swap orphelin dans `initramfs-tools/conf.d/resume`.
- **Toujours vérifier individuellement chaque noyau après une régénération groupée d'`initramfs`** — une commande `update-initramfs -u -k all` peut, dans de rares cas, omettre le noyau le plus récent.
- **Distinguer les erreurs bloquantes des avertissements bénins** pendant une mise à jour apt (ex. erreurs D-Bus, échec de redémarrage d'un service non configuré, échec de résolution de nom d'hôte en chroot) — ne pas interrompre une opération en cours sur la seule base d'un texte en rouge sans en vérifier l'impact réel.
- **Toujours simuler avant d'exécuter (`wipefs -n`, `rsync --dry-run`)** pour toute opération destructrice ou de grand volume — cette discipline a permis de valider les volumes attendus avant chaque transfert réel du projet.

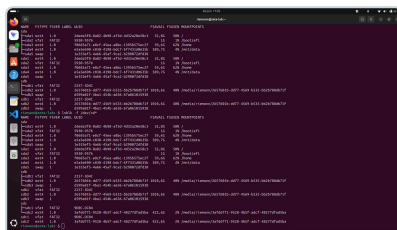
— Fin du corps du rapport — Projet Zêta / Riemann Lab — riemann@zeta-lab —

11. Annexe — Galerie complète des captures d'écran

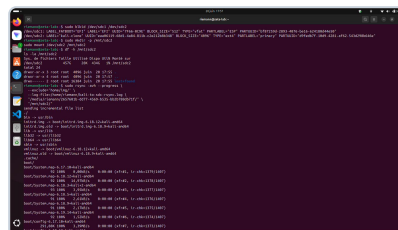
Cette annexe regroupe l'intégralité des 45 captures d'écran disponibles pour ce projet, classées par ordre chronologique. Chaque capture déjà intégrée dans le corps du rapport (§2 à §7) est également listée ici à des fins d'indexation complète et de traçabilité.



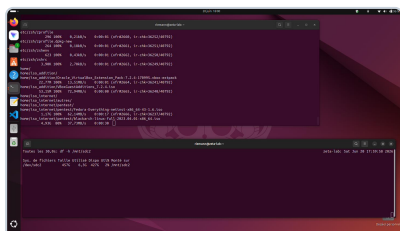
20 juin, ~17:00 — *Isblk initial — Micron/BitLocker détecté par erreur sur sdb*



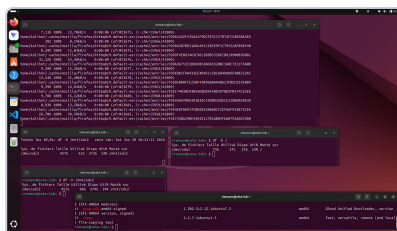
20 juin, 17:20 — *Isblk -f réinventaire correct (sdb=Toshiba, sdc=Samsung)*



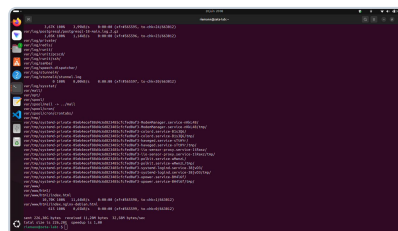
20 juin, 17:57 — *rsync Kali lancé avec sudo, exclude home/img*



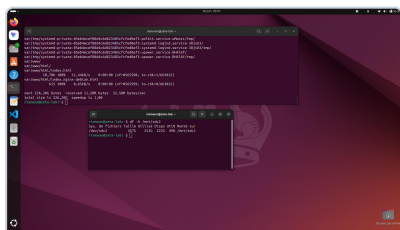
20 juin, 18:00 — *Suivi double-terminal pendant rsync Kali*



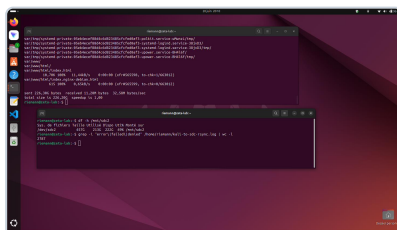
20 juin, 18:31 — *Suivi multi-terminal + vérif paquets rsync/grub installés*



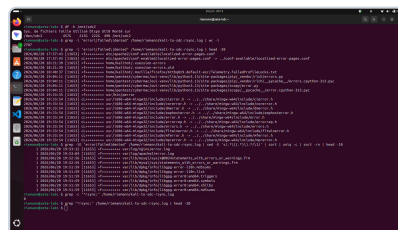
20 juin, 20:08 — *Fin du rsync Kali — 226,30G envoyés*



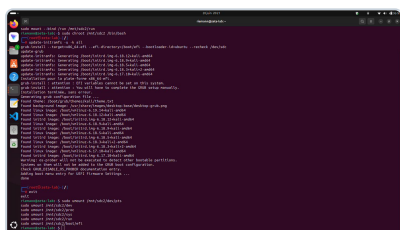
20 juin, 20:10 — *df -h final /mnt/sdc2 (213G/49%)*



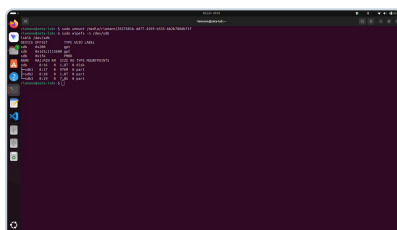
20 juin, 20:10 — *grep error/failed/denied = 2787 lignes (alerte apparente)*



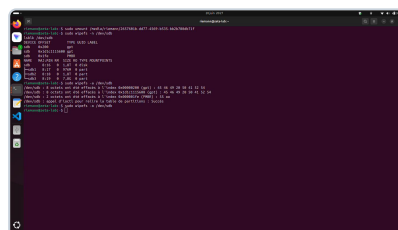
20 juin, 20:13 — *Analyse fine — faux positifs confirmés, 0 vraie erreur rsync*



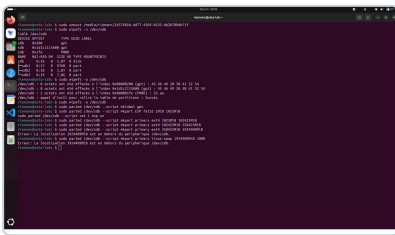
20 juin, 20:21 — *Premier grub-install Kali en chroot (avant --removable)*



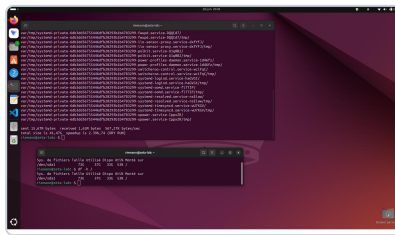
20 juin, 20:23 — *wipecfs -n (dry-run) sur le Toshiba avant formatage*



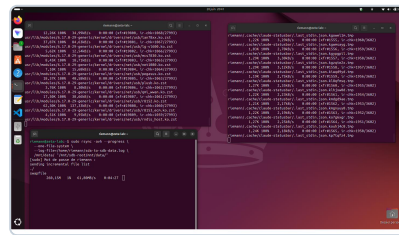
20 juin, 20:27 — *wipecfs -a réel exécuté sur le Toshiba*



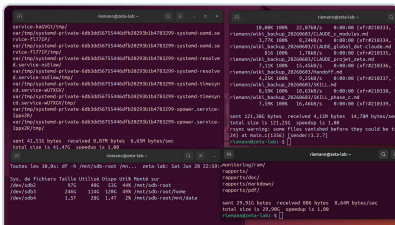
20 juin, 20:28 — Erreur de dépassement de capacité parted (1924409MiB)



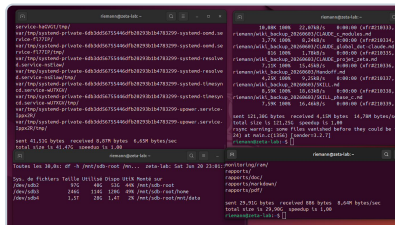
20 juin, 20:38 — rsync --dry-run sur la racine Ubuntu (DRY RUN, 41,47G)



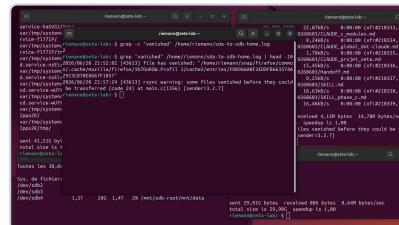
20 juin, 20:41 — Trois rsync en parallèle (/ , /home , /mnt/data)



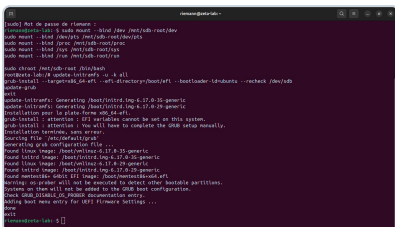
20 juin, 22:59 — Fin rsync /mnt/data + warning vanished + df -h multi-partitions



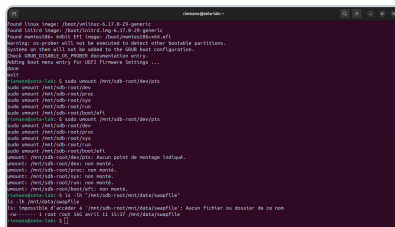
20 juin, 23:01 — Suivi identique (quelques minutes plus tard)



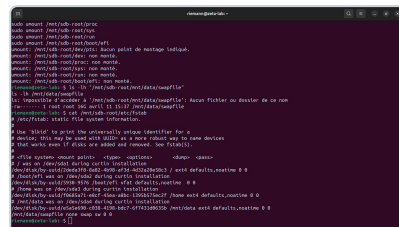
20 juin, 23:07 — Vérification du warning vanished — fichier cache Firefox



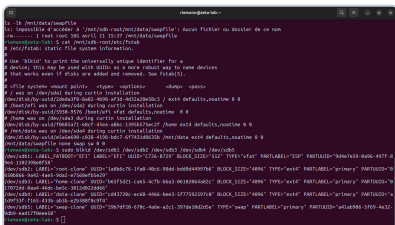
20 juin, 23:11 — Installation GRUB complète en chroot (clone Ubuntu)



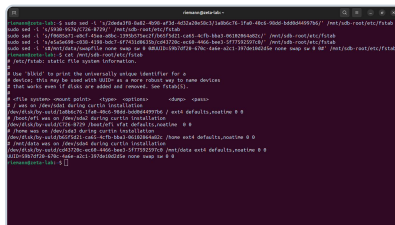
20 juin, 23:13 — fstab du clone Ubuntu AVANT correction (UUID sda)



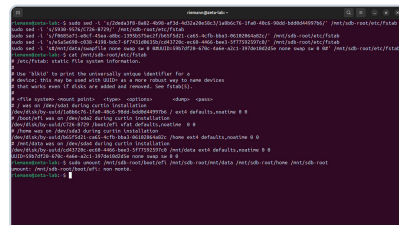
20 juin, 23:13 — blkid des 5 partitions du Toshiba (incl. swap-clone)



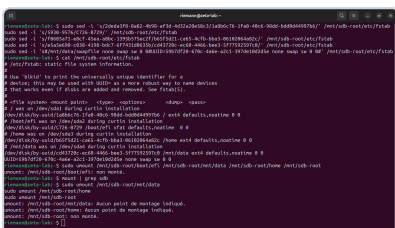
20 juin, 23:14 — sed -i appliqué + fstab APRÈS correction



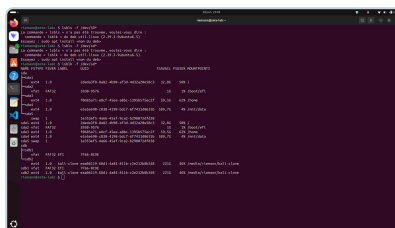
20 juin, 23:15 — Démontage final propre du clone Ubuntu



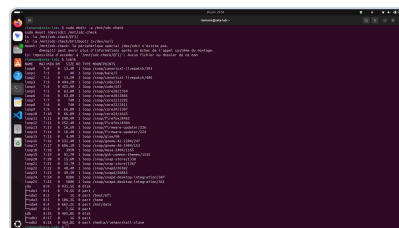
20 juin, 23:16 — Variante du démontage (re-vérification mount)



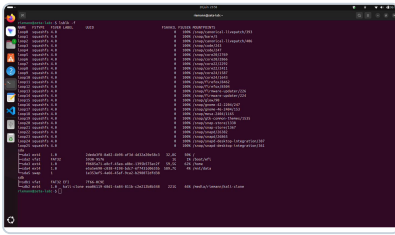
20 juin, 23:17 — Vérification finale mount | grep sdb (vide)



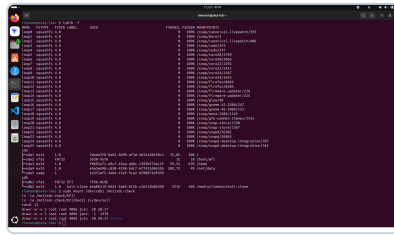
20 juin, 23:49 — Typo lsbls→lsblk corrigée ; Samsung en sdb (kali-clone)



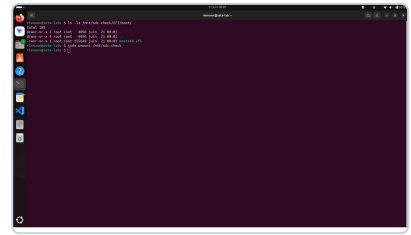
20 juin, 23:56 — Tentative montage /dev/sdc1 inexistant (disque renommé)



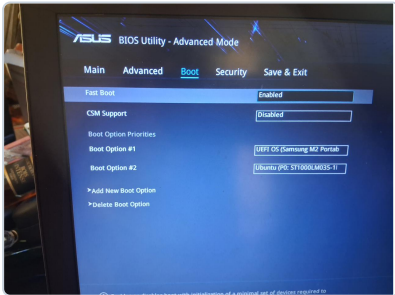
20 juin, 23:58 — Identification correcte par `lsblk -f` (label kali-clone)



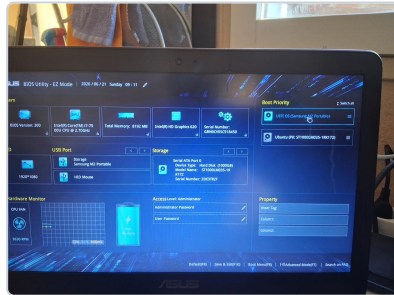
21 juin, 00:00 — Confirmation finale boot Kali — `sdb=kali-clone`



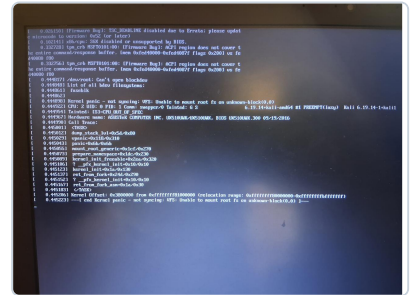
21 juin, 00:02 — Vérification du fallback EFI/boot/bootx64.efi présent



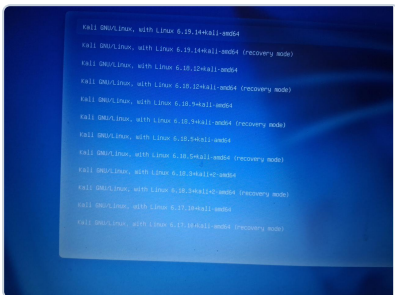
(BIOS) Boot tab — Boot Option #1 = Samsung M2 Portable



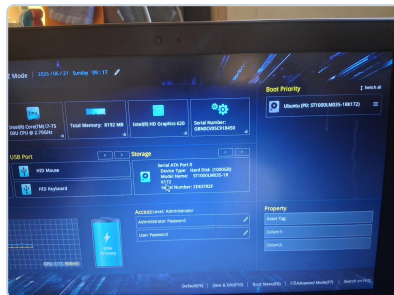
(BIOS) EZ Mode — Samsung confirmé en priorité 1, fallback opérationnel



(boot) Kernel panic — Trace complète VFS unable to mount root fs



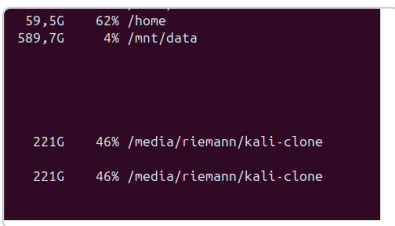
(GRUB) Menu avancé — Liste des 6 noyaux Kali installés



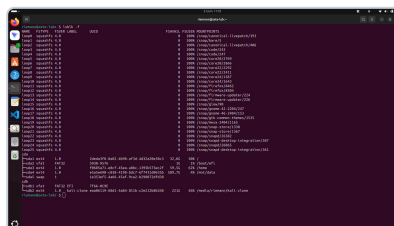
(BIOS) EZ Mode — Samsung disparu de la Boot Priority (incident \$6.5)



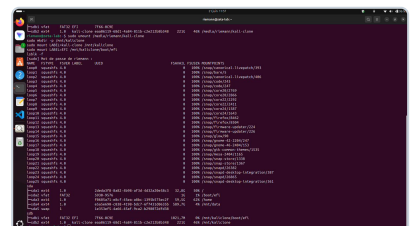
21 juin, 10:56 — Re-vérification `lsblk -f` du matin



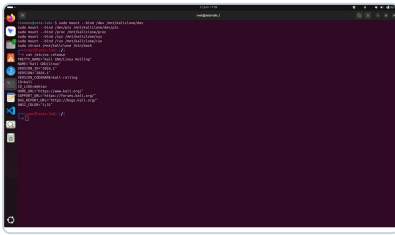
21 juin, 11:09 — Détail recadré du `lsblk` précédent



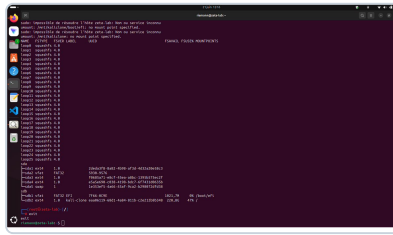
21 juin, 11:55 — Re-vérification `lsblk -f` (session Phase E)



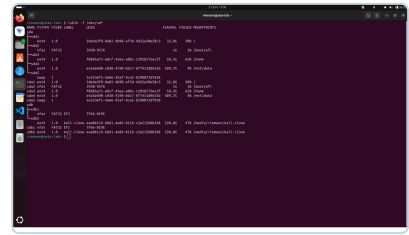
21 juin, 11:57 — Montage par LABEL (kali-clone, EFI)



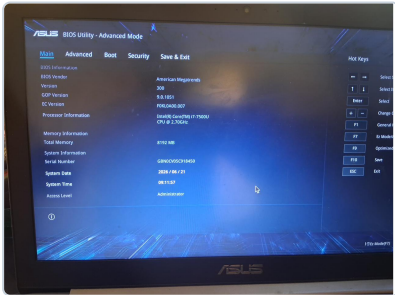
21 juin, 11:58 — *chroot + os-release : Kali 2026.1 Rolling confirmé*



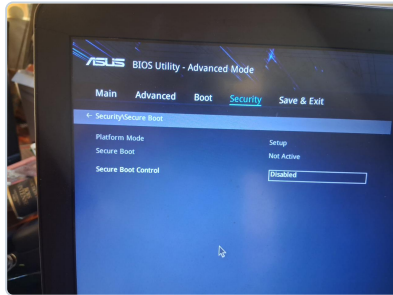
21 juin, 12:16 — *Warning résolution hôte en chroot + démontage complet*



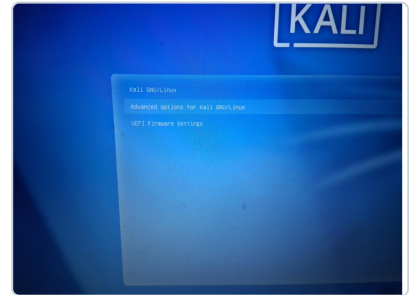
21 juin, 12:38 — *lsblk -f final, propre, après sortie de chroot*



(BIOS) Main — *Informations système — i7-7500U, 8192MB, BIOS v300*



(BIOS) Security — *Secure Boot Control: Disabled*



(GRUB) Menu Kali — *Kali GNU/Linux / Advanced options / UEFI Firmware Settings*